

MANHATTAN SCALE™

WAREHOUSE MOBILE GUIDE

WAREHOUSE MOBILE GUIDE

Copyright and Disclaimer

Copyright © 2026 Manhattan Associates, Inc. All rights reserved.

This documentation, as well as the software described in it, is furnished under license, and may be used or copied only in accordance with the terms of such license. The information in this documentation is furnished for informational use only, is subject to change without notice, and should not be construed as a commitment by Manhattan Associates, Inc. ("Manhattan"). No third-party patent liability is assumed with respect to the use of the information contained herein. While every precaution has been taken in the preparation of this documentation, Manhattan assumes no responsibility for errors or omissions.

EXCEPT WHERE EXPRESSLY PROVIDED OTHERWISE, ALL CONTENT, MATERIALS AND INFORMATION, ARE PROVIDED ON AN "AS IS" AND "AS AVAILABLE" BASIS. MANHATTAN EXPRESSLY DISCLAIMS ALL WARRANTIES OF ANY KIND, WHETHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT.

Except as permitted by license, no part of this documentation may be reproduced, stored in a retrieval system, or transmitted, in any form by any means, electronic, mechanical, recording, or otherwise, without the prior written permission of Manhattan Associates.

Manhattan Associates is a registered trademark of Manhattan Associates, Inc.

All other brands, products, or services are trademarks, registered trademarks, or service marks of their respective companies or organizations.

Contact Address:

Manhattan Associates, Inc.
2300 Windy Ridge Parkway,
Atlanta, Georgia 30339
<http://www.manh.com/>

Region	Telephone	Email
Americas	+1 877.756.7435 (U.S. and Canada only) +1 404.965.4025	callcenter@manh.com
Asia	(00) 800.988.0885 (China only) +86 21 3311 3499	asiacustomersupport@manh.com
Europe, Middle East, and Africa	+44 (0) 1344 318400 (UK) +31 (0)30 214 3400 (NL)	emeacustomersupport@manh.com
Australia	+61 1300787050	aucustomersupport2@manh.com

Unsure whom to contact? Call +1 404.965.4025 and you will be routed to the appropriate support group.

Table of Contents

Introduction	1
Pre-requisites	1
Recommended configuration	1
Not supported	1
Certified Devices	1
Warehouse Mobile configuration	2
Server-side configuration.....	2
Define the SRC file path	2
Screen Flow configuration	3
App Deployment.....	5
Option 1: Managed Configuration	5
Option 2 : Download and Install the APK file manually.....	6
Device logging	11
Configuration	11
View logs	11
Device logs.....	12
Server logs.....	12
Configure Session timeout	13
Architecture.....	15
Menu 15	
Screen Layout	16
Architecture Overview	17
Structure of SRC – JSON file	18
Properties of a Flow	18
GO button reference	18
Actions	19
Control with attributes	20
Warehouse Mobile Buffer scanning	54
Activate Buffer Scanning.....	55
Buffer Queue Size.....	55
Warehouse Mobile Progress Bar loader	56
Success Message Duration.....	57
Warehouse Mobile Multi-Segment Barcode	58
Enabling the Feature.....	58
Configuration.....	58
How It Works.....	59
API Response Structure	60
BarcodeData Application in Generic View Processing	61
Date Format Handling (GS1 Barcodes)	61
Error Handling	61
BarcodeData Lifecycle	62
Supported Control Types	62
Warehouse mobile custom SRC	63
Configuration.....	63

WAREHOUSE MOBILE GUIDE

Custom SRC format..... 63



Introduction

Warehouse Mobile application provides a comprehensive mobile application framework and device aware platform which enables the application to run on modern mobile Android devices. This application will eventually replace the Manhattan SCALE RF Android application. This document outlines the installation and configuration of the mobile application.

The application's layout and screen flow are driven by JSON files. This allows for easy customization of the application. See the SDK for further information on the architecture of these files. Warehouse Mobile is a modernized app for Android devices, which will eventually replace the RF functionality of today.

Pre-requisites

- The mobile device should be connected to the network.
- Manhattan SCALE should be configured on the application server.
- The Manhattan SCALE applications web server should be configured with the well-known certificate.
- To make customizations a basic understanding of web technologies is required.

Recommended configuration

Manhattan SCALE Warehouse mobile application only supports the Android OS platform.

Device Requirements
Android 8 or 8+
Android WebView 149 or above
2GB+ RAM

Not supported

- AD FS Windows Integrated Authentication (WIA) is not supported with Warehouse mobile. See AD FS Single sign-on guide for more information on alternative configurations.

Certified Devices

See the Release notes for more information on the certified devices for Warehouse Mobile.

Warehouse Mobile configuration

Server-side configuration

The following server-side configuration steps must be completed before accessing the Warehouse Mobile application.
Follow the steps below:

Define the SRC file path

Manhattan SCALE ships with SRC files in the installation location:
C:\ManhattanAssociates\ManhattanScale\ *Screen Flows*. In this step, you can point the configuration to the default screen flow or a path with a custom screen flow. Make sure to back up any custom files to ensure they are not lost on upgrades.

- a) Log in to the SCALE Desktop application.
- b) Navigate to Configurations | Technical | Technical values.
- c) Open the SRC File path record.
- d) In the system, the value field enters the SRC File path stored in the system.

Technical values - Edit existing

File Actions Help

Key: 440

Description: SRC file path

System value: \\<server_name>\JLS\Screen Flows

☐ Value required

User defined data

User defined field 1:		User defined field 2:	
User defined field 3:		User defined field 4:	
User defined field 5:		User defined field 6:	
User defined field 7:	0.00000	User defined field 8:	0.00000

☒ System created

OKCancel

WAREHOUSE MOBILE GUIDE

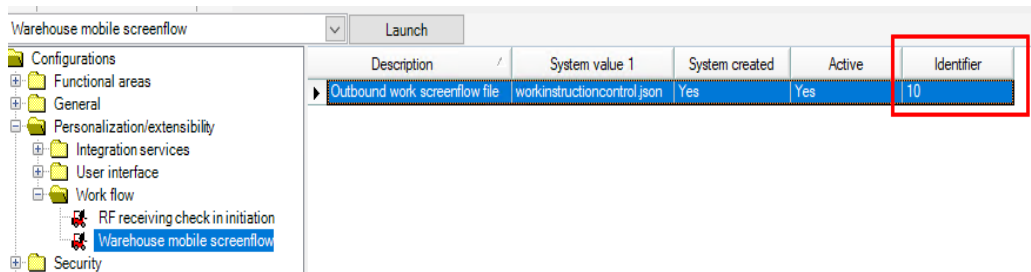
Screen Flow configuration

The next step is to configure the Warehouse Mobile Screen flow records. By default, the SRCs will be configured for each flow, if this needs to be customized to use different SRC, the same can be configured.

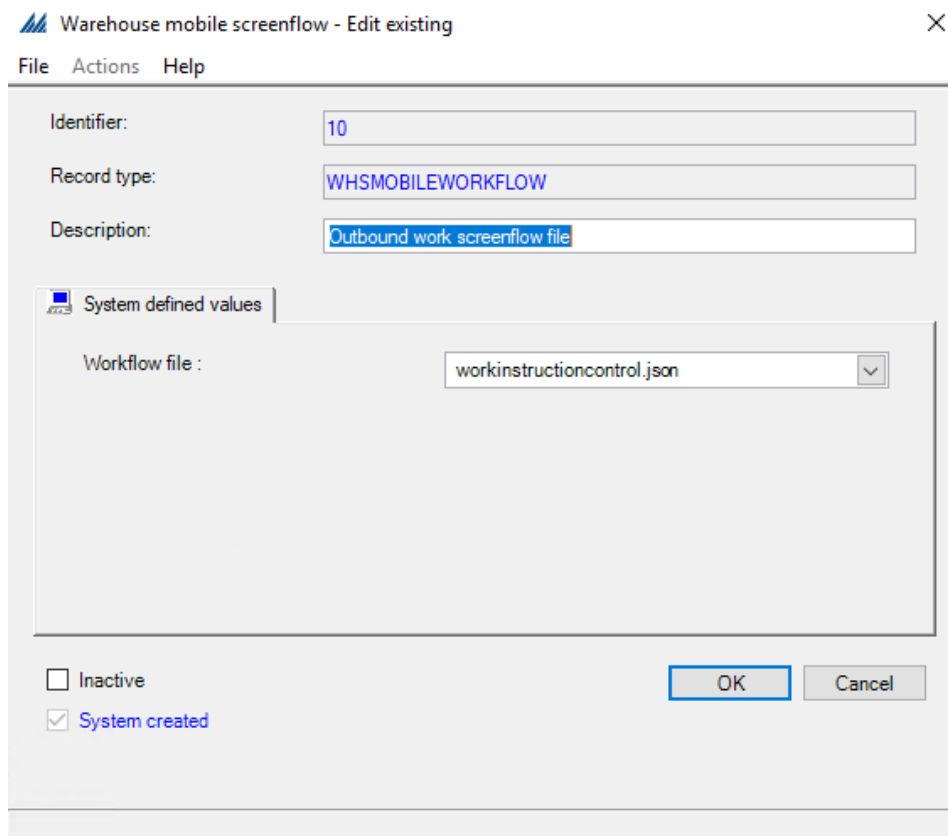
This needs to be done in Configurations -> Personalization/extensibility -> Work flow -> Warehouse mobile screenflow

1. Outbound work

- a) Open the Outbound work screenflow file record. For the default record that is shipped this value is 10.



- b) Next, select the workflow file from the dropdown and click OK to save the changes.



WAREHOUSE MOBILE GUIDE

2. Receiving – Header Item

- Open the Receiving record, the identifier for the default Receiving record for header-item initiation is 30.
- Select the workflow file from the dropdown and save the changes.

Warehouse mobile screenflow - Edit existing

File Actions Help

Identifier: 30

Record type: WHSMOBILEWORKFLOW

Description: Receiving

System defined values

Workflow file : Receiving.json

Functional area : Receiving

☐ Inactive
☒ System created

OK Cancel

Last updated: 11/20/2019 4:06:09 AM User: System

3. Receiving – License plate

- Open the Receiving with initiation via license plate record, the identifier for the default Receiving record for license-plate initiation is 50.
- Select the workflow file from the dropdown and save the changes.

Warehouse mobile screenflow - Edit existing

File Actions Help

Identifier: 50

Record type: WHSMOBILEWORKFLOW

Description: Receiving with initiation via license plate

System defined values

Workflow file : ReceivingLicensePlate.json

Functional area : Receiving

☐ Inactive
☒ System created

OK Cancel

Last updated: 3/2/2020 7:30:28 AM User: System

WAREHOUSE MOBILE GUIDE

4. Receiving – Blind

- a. Open the Receiving Blind record, the identifier for the default Receiving record for blind initiation is 330.
- b. Select the workflow file from the dropdown and save the changes.

Warehouse mobile screenflow - Edit existing

File Actions Help

Identifier: 330

Record type: WHSMOBILEWORKFLOW

Description: Receiving Blind

System defined values

Workflow file : ReceivingBlind.json

Functional area : Receiving

☐ Inactive

☒ System created

OK Cancel

Last updated: 6/3/2022 8:19:13 AM User: System

App Deployment

This section will walk through the installation of the application on the Android device. There are two options

Option 1: Managed Configuration

This configuration works when your device is registered under any Mobile Device Management system (for Example SOTI)

1. Navigate to the MDM policies portal.
2. Create a new policy for Android enterprise.
3. Add apps apk to the policy (App apk can be found under folder %SCALE_HOME% on SCALE Server)
4. Enable Managed app configuration.

WAREHOUSE MOBILE GUIDE

ADVANCED CONFIGURATIONS

Configuration Options

App Details

Installation Options

Managed App Config

Specify the configuration options for the application you are adding or editing

Enable Managed App Config

Advanced options available for the application

☐

SCALE Server FQDN

This is host name where the SCALE server is running
ex - scale.xyz.com

scaleserver.manhdev.com

Device Log level- Default is Error

INFO

▼

server Log level- Default is Error

INFO

▼

Play sound on error

☐

Play sound on success

☐

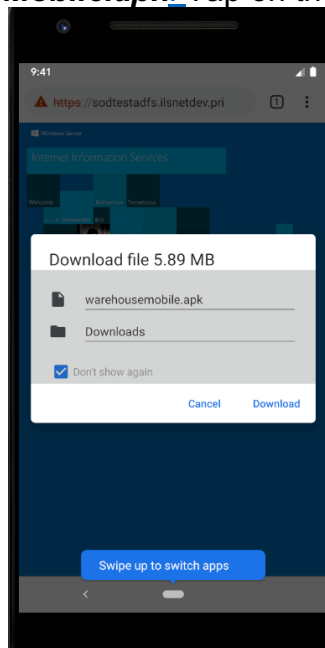
CANCEL

SAVE

Option 2 : Download and Install the APK file manually

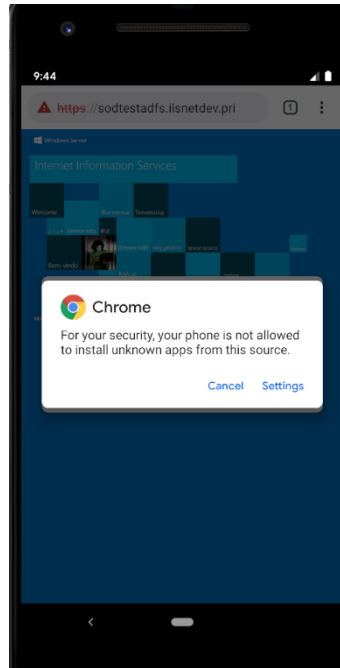
Follow the steps to download and install the APK

To download the APK file to the Android mobile device, the user needs to browse to the following URL on the device browser via ***https://<hostname>/WarehouseMobile/WarehouseMobile.apk***. Tap on the downloaded APK file to install.

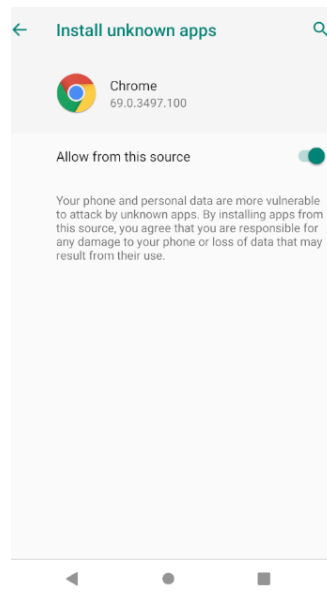


1. Since the APK is not being downloaded from the Android play store you will need to modify an additional setting to install this apk. Click **Settings**.

WAREHOUSE MOBILE GUIDE

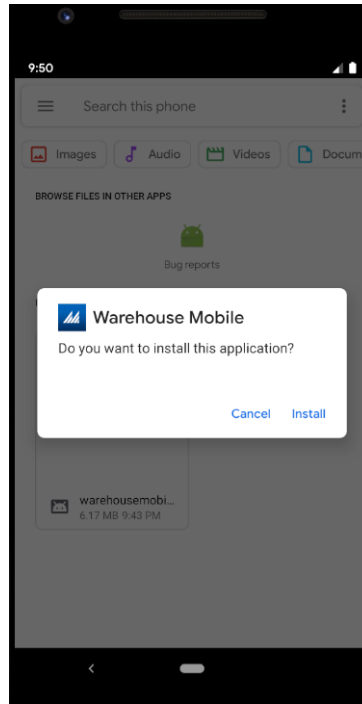


2. Turn on Allow apps from this source.



3. Click **Install**.

WAREHOUSE MOBILE GUIDE



4. A config.json file will need to be created with the host server information that Warehouse Mobile should access.
 - For Android 10 and above devices support scoped storage, user would need to create config.json file under **sdcard/Android/media/com.manh.scale/whsmobile** folder.
 - For Android 9 and older versions create the config.json file on the device under the **sdcard/download** folder.

Note:

- Clear Storage on WM app deletes folder "whsmobile"
- Uninstalling WM App deletes folder "com.manh.scale"

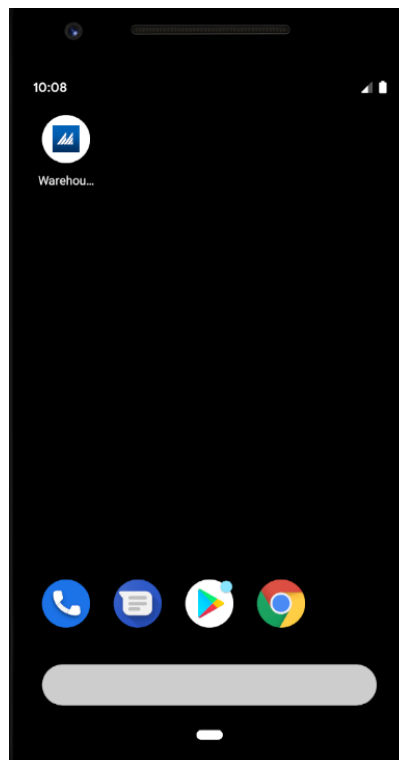
```
{
  "host": "prod scale server host name",
  "stagehost": "stage scale server host name",
  "logconfig": {
    "level": "5",
    "serverLogLevel": "5"
  },
  "successsound": true,
  "errorsound": true
}
```

WAREHOUSE MOBILE GUIDE

- a. Specify the prod scale server's name for the host if no environments are configured. The server's name should be specified in the fully qualified domain name of the prod server. For example, server.mydomain.com.
- b. Specify the environment name for the host if environments are configured. The environment name should be specified in the fully qualified domain name of the prod server with the environment name as subdomain. For example, env1.mydomain.com
- c. Specify the stage scale server's name for the stagehost if no environments are configured. The server's name should be specified in the fully qualified domain name of the stage server. For example, stageserver.mydomain.com.

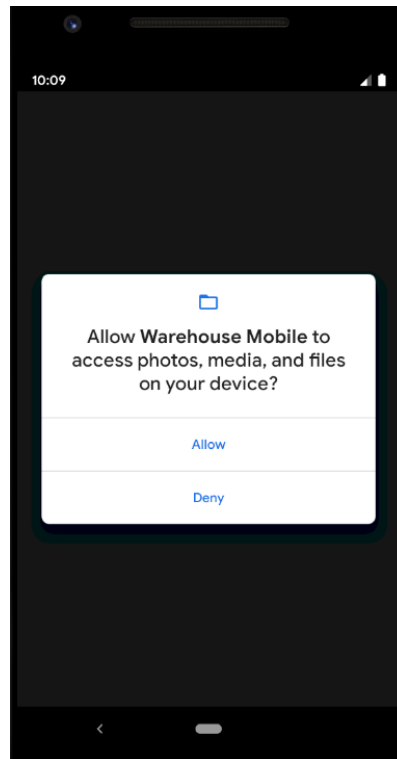
Note: If device is Android 10 and above, the config.json file needs to be placed on the given path **sdcard/Android/media/com.manh.scale.stage/whsmobile**

- d. Specify the environment name for the host if environments are configured. The environment name should be specified in the fully qualified domain name of the server with the environment name as subdomain. For example, env1stage.mydomain.com
 - e. logconfig attribute is discussed in [Device logging](#) section.
5. Open the Warehouse Mobile app on the device.

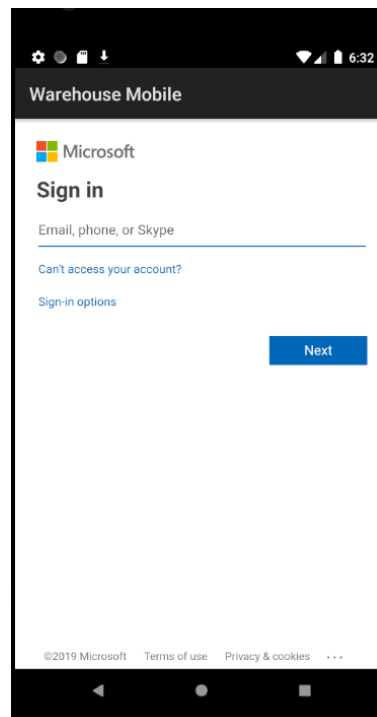


6. Select Allow to give the application access to storage on the device. This is required for the application to read the host name from the config.json file.

WAREHOUSE MOBILE GUIDE



7. When the application is successfully installed the login screen will appear. Specify your credentials to access the Warehouse Mobile application.



Device logging

This section walks through Warehouse Mobile logging capabilities and configuration.

Configuration

With logging configuration, you can set what logs you want to capture on the device or server. Warehouse Mobile logging configuration which goes in config.json looks like this

```
"logconfig": {  
  "level": "<loglevel>",  
  "serverLogLevel": "<loglevel>"  
}
```

1. **level** - This indicates the level of logs to be captured on the device Android Debug Bridge(adb). For example, if you want to capture Info logs, on the device the config. JSON level attribute looks like "level": "2".

These are different logging levels you can set.

```
TRACE = 0,  
DEBUG = 1,  
INFO = 2,  
LOG = 3,  
WARN = 4,  
ERROR = 5,  
FATAL = 6,  
OFF = 7
```

2. **serverLogLevel** - This indicates what level of logs are to be captured and sent to the server. For example, if you want to capture Error logs, on the device the config. json level attribute looks like "serverLogLevel": "5".

Note- If logging configuration is not present in config. JSON then will use the default logging configuration which is set to Info level for device logs and error for server.

View logs

This section talks about how to view logs captured for Warehouse Mobile.

WAREHOUSE MOBILE GUIDE

Device logs

ADB logs can be viewed using device inbuilt log management applications like

- RXLogger for Zebra devices with logcat module. <https://techdocs.zebra.com/rxlogger/5-4/guide/about/>.
- HXLogger for Honeywell devices with logcat module. <https://support.honeywellaidc.com/s/article/What-are-the-logging-options-on-a-Honeywell-Android-device>
- Remote/wired adb using logcat commands. Example **`adb logcat chromium:D SystemWebViewClient:D *:S`**

Server logs

Device logs are sent to the server and logged using configuration under the logger name WMDevice Which is shipped into log4net config and default to info.

- The log level enabled for the above-mentioned name space correspondingly log the logs.
 - For example, if the WM application is sending the debug logs but the WMDevice is set to info (which is by default enabled) then it will not log the debug log. You may need to enable the debug logs for the WMDevice. This gives the control to the scale server for what to log and what not to.

```
<logger name="WMDevice" additivity="false">
  <level value="Info"/>
  <appender-ref ref="WMDeviceeeAsyncBufferingForwarder"/>
</logger>
```

-
- If SCALE is configured with Opensearch then the logs, go to Opensearch with indices wmdevicelog
- If SCALE is not configured with Opensearch then logs, go into logfiles as per the log4net configuration.

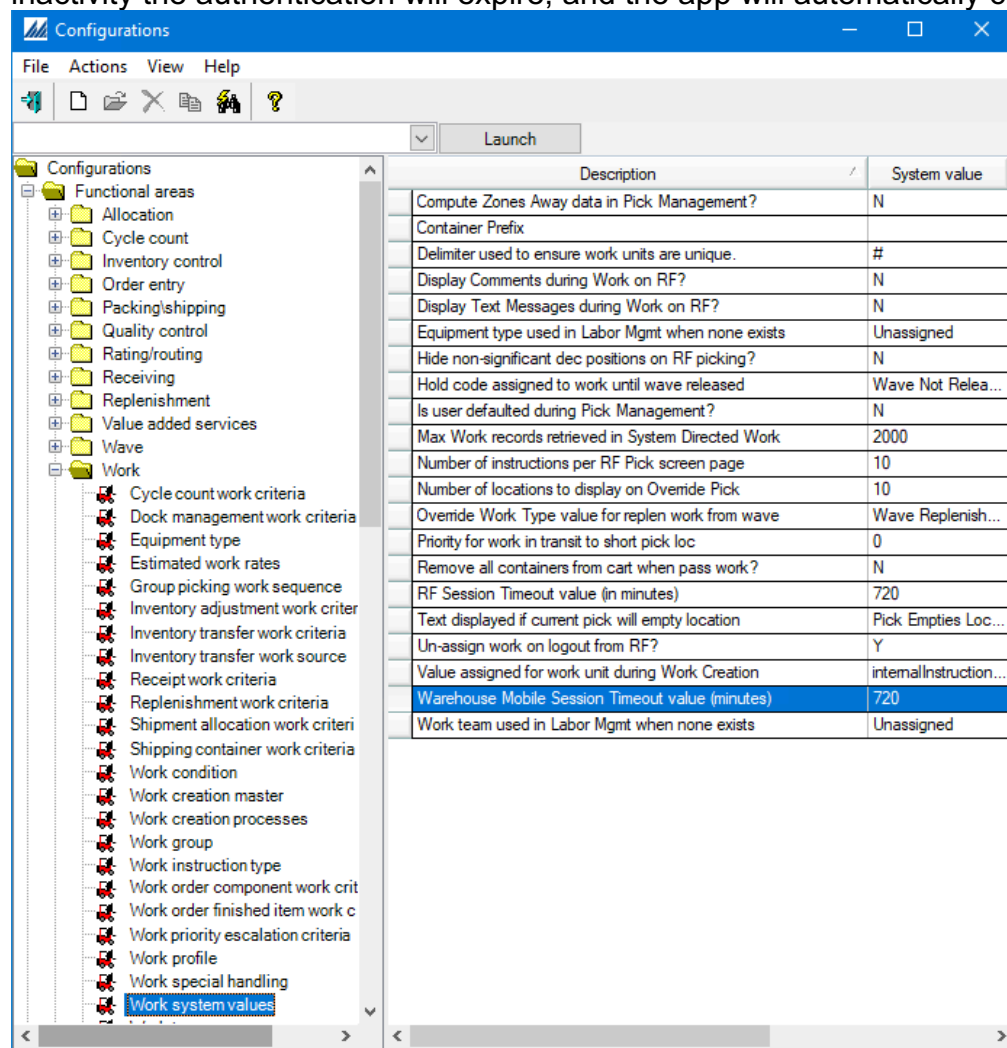
Note- Use a centralized logging infrastructure like Opensearch is recommended for viewing of server logs.

Configure Session timeout

Warehouse mobile supports session timeout.

If no action is made on the device, then logged user authentication will expire and the Warehouse Mobile app will close after a specified interval defined in Scale Configuration / Functional areas / Work / Work system values: “Warehouse Mobile Session Timeout value (minutes)”.

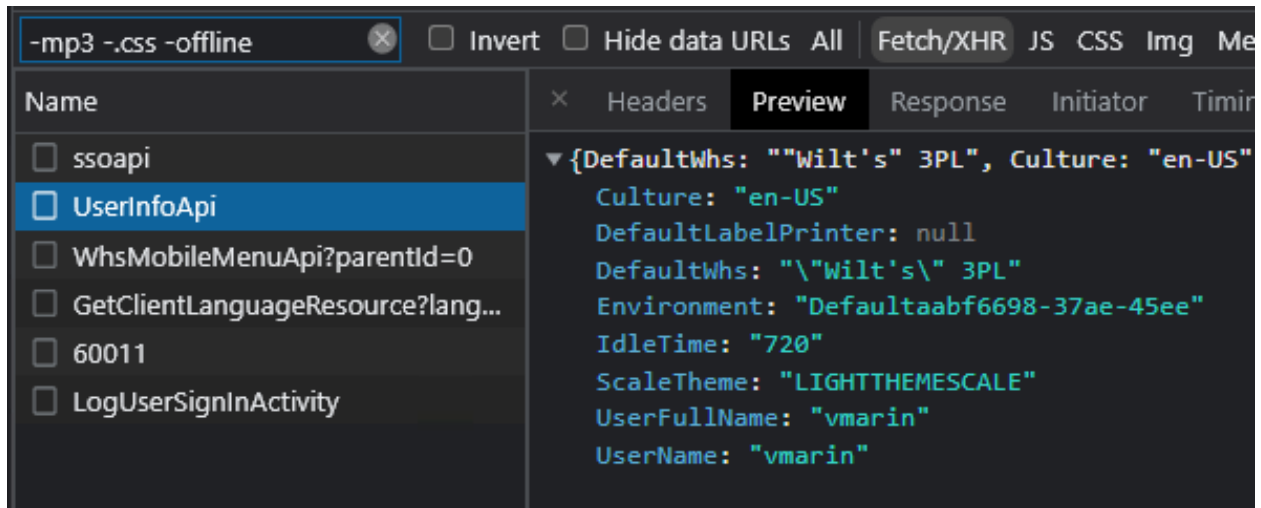
By default, this value is set at 720 and which means that after 720 minutes (12 hours) of inactivity the authentication will expire, and the app will automatically close.



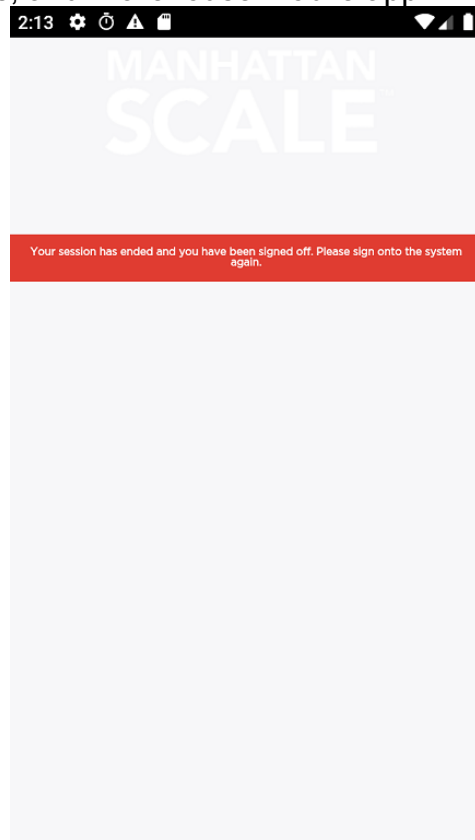
This value is defined as part of UserInfoApi and the identifier for it is: “IdleTime”.

The call of “UserInfoApi” is executed each time the warehouse mobile app is started. If “Warehouse Mobile Session Timeout value (minutes)” (Idle Time) is modified in the database from the scale configuration window, then “Idle Time” can be updated if the user signs out and then sing-in again, or app is closed and reopened.

WAREHOUSE MOBILE GUIDE



When user authentication expires, on the device screen the next message will appear for 3 seconds, and Warehouse mobile app will close.



Architecture

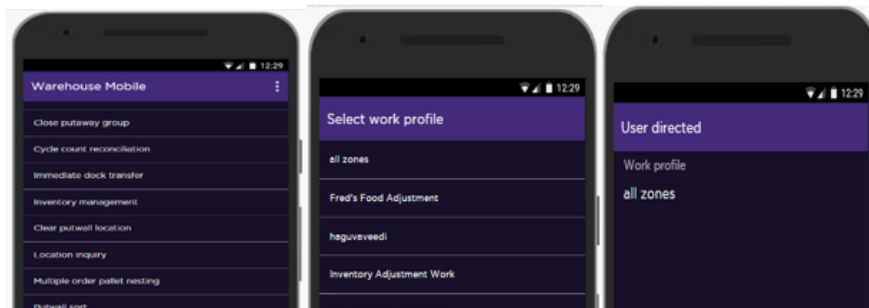
Below is a high-level overview of the Warehouse Mobile architecture. See the Activity Architect User Guide for more information on customization.

Menu

- Warehouse mobile gives configurable menu capabilities.
- Menu are saved as a tree structure in WAREHOUSE_MOBILE_MENU table.
- Records with **parent object id** null are treated as top-level menu.
- Form ID is required to provide license and security validation.
- **Screen Name** and **Screen Resource Key** are used for sub screen title.
- **Submenu authorization** should be null by default and the base provide three options for this column to provide the mapping between the menu and existing configuration like work profile, adjustment type and receiving preference. This also provides default functionality like if the user is assigned to a default profile, then load that submenu automatically on the screen.
 - **workprofile** – This validates if sub menu option name should exist as a work profile and the user is authorized to access that profile.
 - **adjustmentprofile** – This validates if sub menu option name should exist as an adjustment profile and the user is authorized to access this profile.
 - **receivingprofile** - This validates if sub menu option name should exist as receiving preference, the user is authorized to access that preference and the preference receiving dock exists in the user's current warehouse.
- **Src identifier** is the identifier for src file to load when the menu is selected. If this is null, then the screen will try to drill down as submenu.

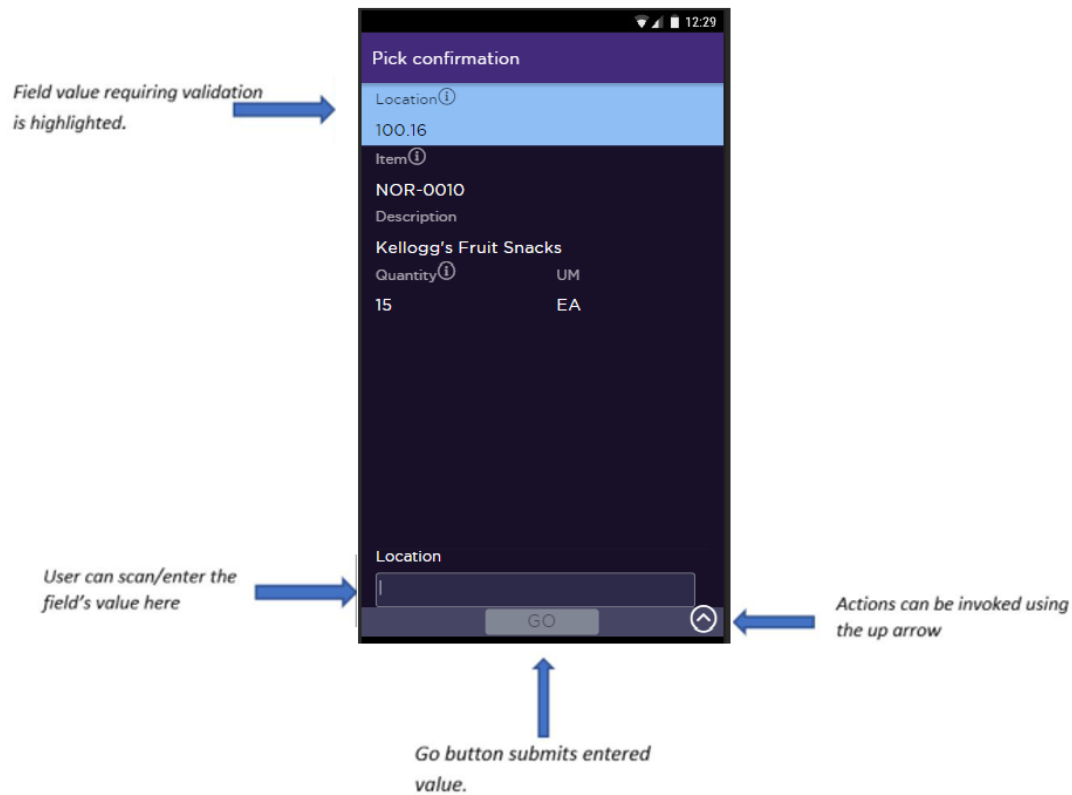
Example configuration

	OBJECT_ID	MENU_OPTION_NAME	MENU_RESOURCE_KEY	SCREEN_NAME	SCREEN_RESOURCE_KEY	FORM_ID	SEQUENCE	SUBMENU_AUTHORIZATION	SRC_IDENTIFIER	PARENT_OBJECT_ID
1	12	Work	UI_WMWORKEXECUTION	Select work Profile	SELECTWORKPROFILE	60010	NULL	NULL	NULL	NULL
2	13	Close Container	UI_WMCLOSECONTAINER	NULL	NULL	60020	NULL	NULL	50	NULL
3	14	Receiving	UI_WMRECEIVING	Select receiving preference	SELECTRECEIVINGPREFERENCE	60021	NULL	receivingprofile	NULL	NULL
4	15	Close putaway group	UI_WMCLOSEPUTAWAYGROUP	NULL	NULL	60022	NULL	NULL	40	NULL
5	16	All Zones12	NULL	NULL	NULL	NULL	NULL	workprofile	NULL	12
6	17	MN Blind	NULL	NULL	NULL	NULL	NULL	NULL	300	14
7	18	Adjustment	ADJUSTMENTTYPE	Select adjustment Type	SELECTRECEIVINGPREFERENCE	NULL	NULL	adjustmentprofile	NULL	NULL
8	19	Status Change	StatusChange	NULL	NULL	NULL	NULL	NULL	NULL	18
9	24	All Zones	NULL	NULL	NULL	NULL	NULL	workprofile	10	16
10	25	Adjustment1	ADJUSTMENTTYPE	Select adjustment Type	SELECTRECEIVINGPREFERENCE	NULL	NULL	adjustmentprofile	NULL	NULL



WAREHOUSE MOBILE GUIDE

Screen Layout

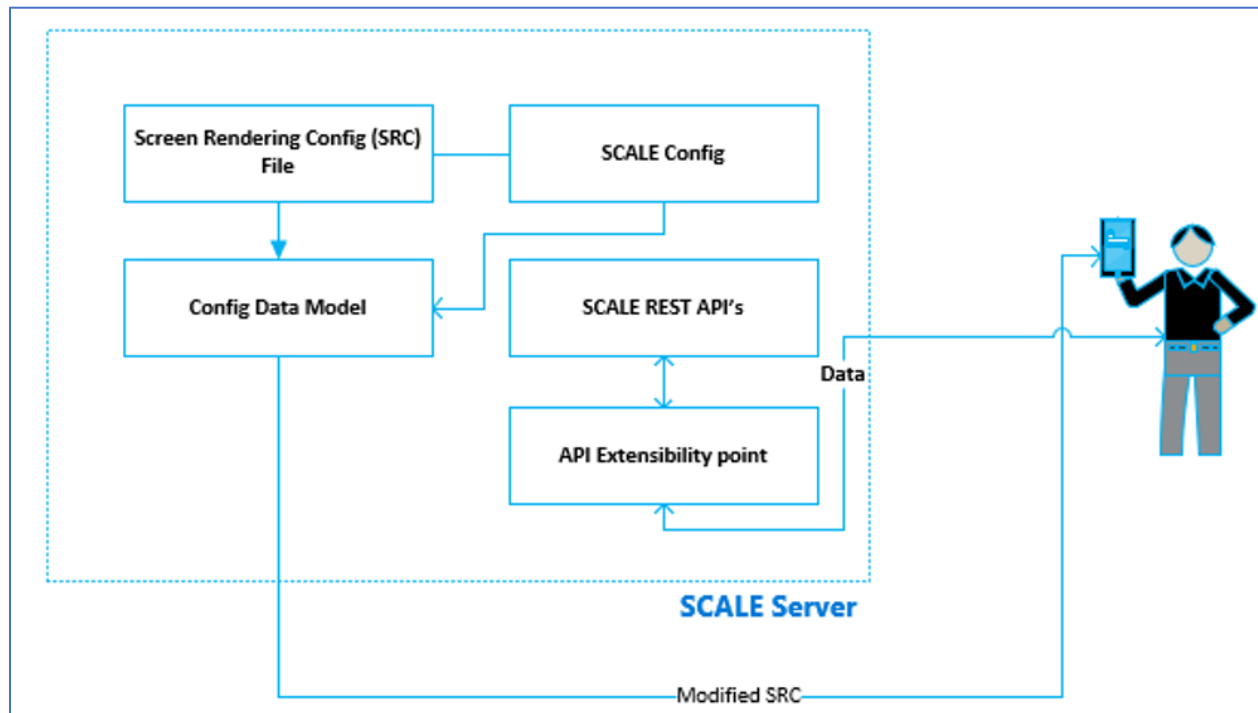


As seen above all screens in Warehouse Mobile have a consistent look and feel as been by each screen meets the following criteria:

- Only one input field on a screen at a time, always at the bottom
- The field requiring validation is highlighted in blue.
- The action menu is always on the bottom lower right of the screen defined by the up-arrow icon.
- Fields may have information buttons that provide more details about an item or location for example.
- The back button will appear in the left corner when applicable.

Architecture Overview

The Warehouse mobile application renders the UI based on a JSON file called the Screen Rendering Configuration (SRC) file. The main components of the application are as defined below.



- **Screen Rendering Configuration (SRC) File** (JSON) - The SRC file is used to render the UI based on its content.
- **Data Model** - The data control system which drives the screen information details and flow.
- **Config Data Model** - This represents the configuration details and security layers of the SCALE application.
- **API Extensibility point** –
 - **Data Model** - Custom modification of the data model can be done using these extensibility end points which are configured as Web API exit points.
 - **UI** – Custom UI modification can be done by modifying the SRC file. **Note**- Never change the base SRC file directly, always modify the copy of the base SRC and keep it in your custom content delivery folder. This will ensure the file is not lost when applying updates or upgrading the application.
- **Warehouse Mobile Endpoint** - Warehouse Mobile is enabled to call custom Azure functions. API “whmendpoint-executed” supports this behavior. Check “Warehouse Mobile Endpoint” under Extensibility in SDK for more details.

Structure of SRC – JSON file

This section will walk through the structure of the SRC JSON file. Keep in mind that in the Warehouse Mobile application, A single screen is shown on the device, or a browser may contain one or more pages, in other words, A screen is a collection of pages and an SRC file can define many screens.

- An SRC file contains initialDataModel to load the initiation screen.
- An SRC file may contain initialConfigDataModel to load the screen's default behavior. For example, see receiving SRC.
- An SRC file may contain defaultConfigModel to load default values if the config data model does not supply property values. For example, see receipt container nesting SRC.
- A single SRC file contains an array of **Screen Flows**
- A **Flow** is a collection of **Pages** for a single screen.
- A **Page** is a collection of **Controls, Actions, and a GO button reference**
- A **Control** defines the DOM elements (textbox, radio button, dropdown etc.,) and their characteristics.

Properties of a Flow

Property	Description
flowName	Flow name defines the current screen name.
Title	The name of the screen is displayed; the value should always be a resource key.
ContinueToNextFlow	Boolean value. • True: When all the control values are prefilled, the screen is automatically executed on behalf of the user • False: Even though the control values are prefilled, the screen waits for the user to confirm the execution.
OnBack	Specifies the page to navigate back when the user clicks on the back button.

GO button reference

Every page must have a Go button reference. The attributes of the Go are:

Property	Description
pageNumber	Specifies the page number
uri	The value specified should be an API path

WAREHOUSE MOBILE GUIDE

	<i>Note: In the current framework the URI of the final page is used.</i>
nextFlow	Specifies the next flow that should be selected from the SRC file after the successful execution.
LoopNextStepFlow	Loops the screen based on the value specified
runValidation	Specifies if client-side validation should execute
validationuri	The value provided should be an API path, this uri is used in combination with runValidation flag

Actions

An **Action** provides the ability for a page to interact with the server, also an action can contain many controls.

A page can contain one or more actions.

The attributes of an Action are

Type: Defines the type of action. A Type can have the following values:

- **MODIFYMODELDATA:** This value is used when the values of the DataModel are to be changed.
- **MODALDIALOG:** This value is used to open a Modal screen.
- **Back:** This value is used to step back a page.

Dialog: This attribute should be is used when the type is MODALDIALOG and it should contain the list of controls shown on the modal screen.

ActionModelData: This attribute should be used when the type is MODIFYMODELDATA and it should contain the values that are to be changed in the DataModel.

nextFlow: This attribute should be used when the type is MODIFYMODELDATA and it contains the flow name of the next screen on successful execution of the current action.

URI: Specifies an API and the API is called when the action is selected.

Enable: The value of the Enable attribute is always preceded by “@” and followed by a config value. The config value is specified in the ConfigDataModel received from the server.

LoopNextStepFlow: Loops the next screen based on the screenflow specified here on click of the action.

Position: This decides the fixed position of actions in the action menu. This property accepts a number (Integer or decimal) value.

- If “Position” property is not present on any of the action objects in the SRC, all the actions are shown in alphabetical order.
- If this property is present on some of the actions in the array, then the actions that don’t have the property will be displayed on the top in alphabetical order then the actions that have “Position” property will be ordered based on the value of the property.
- If all the actions in a page have “Position” property, then they will be ordered based on the value of the property.

WAREHOUSE MOBILE GUIDE

- If a non-number value is passed to “Position” in an action, it will be ordered in alphabetical order at the top.

Control with attributes

An example of control defined in the SRC is shown below. *Note: All the attributes are case sensitive.*

```
{
  "controltype": "textbox",
  "label": "PICKQTY",
  "type": "number",
  "name": "EnteredQuantity",
  "value": "",
  "placeholder": null,
  "required": false,
  "validations": {
    "required": true,
    "pattern": "",
    "minLength": "",
    "maxLength": "20",
    "rangeStart": 0,
    "rangeEnd": "@WorkExecutionChildEntities[0].FromQty"
  },
  "ShowOnAllPage": "true",
  "order": 1
},
```

Attributes

Attribute	Type	Description
controltype	string	A controltype type defines the DOM element to be shown on the browser or the device (textbox, label, dropdown etc.,).
label	string	Name of the control that is displayed and it should always be a resource key.
type	string	A type defines a characteristic of a DOM element. Values allowed are text and number.

WAREHOUSE MOBILE GUIDE

name	string	An important attribute is required to map the control with the DataModel coming from the server.
order	int	Defines the order number of controls on a page.
validations	object	Contains different types of client-side validations on a DOM element.
visible	string	Makes the control visible on the screen if it is set to "true".
textColumn	string	Used for array-based controls as Picker to specify which column to be displayed from the value array (ex. Identifier or Description)
EnableAppend	boolean	It is an optional control attribute that is used for appending multiple scan values in textbox control. Whenever this flag is set to true the user-entered text will not get selected on any async validation error or execute error.
startLaborTimer	boolean	This optional control attribute in the textbox control refreshes the timer in the ProcessStartDateTime request header. When this flag is set to true, the user can track the updated process start time.
truncate	boolean	It is an optional control attribute that is used to truncate the value provided in textbox control. When the truncate attribute is set to true for any controltype, and either a maxLength is defined in the validations or a field length is specified in the resource files base table in the database, the input value in the warehouse mobile textbox will be automatically truncated to the specified length. Truncate attribute only works for input textbox control. To disable this behavior, set the truncate property to false. If both the Truncate and EqualValidate attributes are used together, the truncate attribute will not function and when EnableAppend and Truncate

WAREHOUSE MOBILE GUIDE

		attributes are used together, EnableAppend will not function.
--	--	--

Validations object properties

Property	Type	Description
required	boolean	If true, the value for control must be entered. If false, value does not need to be entered.
pattern	string (Regex)	If specified, the value of the control should match the regular expression.
minLength	string	Indicates the minimum value a control can take.
maxLength	string	Indicates the maximum value a control can take.
rangeStart	number	If the control type is a numeric input box, it specifies the minimum value that should be entered.
rangeEnd	string	This works in connection with rangeStart. The value should be enclosed in double quotes, to map the rangeEnd value with the DataModel use "@" as the prefix.
asyncValidator	Object	asyncValidator is an object that is used for validating a field on the fly using GET or POST API call.

Asynchronous Validation

Validate a control asynchronously by adding a GET or POST API call to the control object in the SRC.

Below are the examples,

Usage:

```
"Controls":[
  {
    ...
    ...
    ...
    "validations": {
      "asyncValidator":{
        "uri" : "<Specify the uri here>",
        "model":{
```

WAREHOUSE MOBILE GUIDE

Rules to be followed:

- “asyncValidator” property should always be present inside the “validations” property.
- “uri” property is a must when asyncValidator property is mentioned.
- “model” is an optional field and the model field should be used only when the API method is POST.
- “model” is a dictionary containing key-value pairs. The model object will be sent to the server during the API call.
- Each value in the “model” dictionary should correspond to the existing DataModel property. The value in the model dictionary will be replaced with the corresponding DataModel property value before calling the API.
- **Please do not use model property if you are using the GET API call.**

A generic API has been added for easy implementation of async validation:

- "uri": "/general/scaleapi/AsyncValidateApi/GenericValidated?input="
- The model needs to be in following format.

```
"model": {
  "asyncValidationType": "LocationExistsAndNotFrozen",
  "AsyncValidationData": "AdjustmentType"
}
```

- The "asyncValidationType" dictates how data is validated and corresponding logic needs to be added in the "Validate" method of "src\Source\C#\General\BL\AsyncValidationHelper.cs" class.
- The "AsyncValidationData" contains the necessary from the entity data which is needed for async validation. If required, more properties in "Validation Model" class at "src\Source\C#\General\Interfaces\AsyncValidationEntities.cs".

Example for GET:

Item Cross Reference uses a GET API call to validate the item information entered by the User.

WAREHOUSE MOBILE GUIDE

```
"Controls": [  
  {  
    "controltype": "textbox",  
    "label": "ITEM",  
    "type": "text",  
    "name": "Item",  
    "EqualValidate": "@ItemVerify",  
    "validations": {  
      "asyncValidator": {  
        "uri": "/general/items/itemCrossReference/"  
      }  
    },  
    "ShowOnAllPage": "true",  
    "order": 4  
  }  
]
```

Example for POST:

```
"Controls": [  
  {  
    "controltype": "textbox",  
    "label": "LICENSEPLATE",  
    "type": "text",  
    "name": "LogisticsUnit",  
    "validations": {  
      "required": true,  
      "asyncValidator": {  
        "uri": "/general/scaleapi/LocationInventoryApi/ValidateLicensePlate?licensePlate=",  
        "model": {  
          "FromContainerId": "containerId",  
          "ToLocation": "ToLocation"  
        }  
      }  
    },  
    "pattern": "",  
    "minLength": "",  
    "maxLength": "20"  
  },  
  {  
    "visible": "@ReadLPValue",  
    "ShowOnAllPage": "true",  
    "order": 4  
  }  
]
```

Information

The information attribute contains additional information for a particular entity. If a control contains an Information attribute, then SAVE engine places an information icon right next to the control value. With the click of the information icon, the SAVE engine opens an information screen.

The information attribute contains key value pairs. Each key represents a resource key, and each value represents a key in a DataModel.

WAREHOUSE MOBILE GUIDE

The parentNode value corresponds to a key in a DataModel. The corresponding key must be a list containing one or more child nodes where each child node represents a set of properties.

If a parentNode is specified and the parentNode has only one child node in the DataModel, then the information screen will look like below

```
"information": {
  "LOCATION": "FromLoc",
  "ONHAND": "PickLocationEntity.OnHandQty",
  "AVAILABLETOPICK": "PickLocationEntity.AvailableQty",
  "INTRANSIT": "PickLocationEntity.IntransitQty"
},
"parentNode": "WorkExecutionChildEntities",
```

SRC

```
TransportContId: null
▼ WorkExecutionChildEntities: [{,~}]
  ▼ 0: {,~}
    Account: "001"
    Company: null
    ContainerID: null
    ContainerType: null
    ConvertedQty: 1
    ConvertedQtyUM: "CS"
    CycleCount: null
    EndDateTime: "0001-01-01T00:00:00"
    EquipmentLoc: null
    FromCheckDig: null
    FromLoc: "100.16"
    FromLocInvAttributesId: 0
    FromLocationWorkZone: null
    FromQty: 12
    FromTrackContainers: "Y"
    FromVerificationMeth: "None"
    FromWhs: "MN"
    InCheckDig: null
    InTrackContainers: null
    InVerificationMeth: null
    InWorkZone: null
    IncomingPDLoc: null
    InternalContainerNum: 0
    InternalInstructionNum: 40239
    *Transportation: 0000
```

DataModel

If a parentNode is specified and the parentNode has multiple child nodes in the DataModel, then the information screen will show a slider for each child node. The example is shown below.

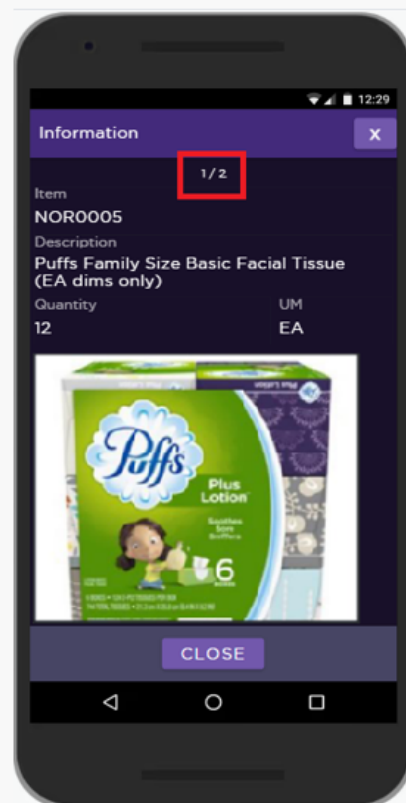
WAREHOUSE MOBILE GUIDE

```
"information": {  
  "LOCATION": "FromLoc",  
  "ONHAND": "PickLocationEntity.OnHandQty",  
  "AVAILABLETOPICK": "PickLocationEntity.AvailableQty",  
  "INTRANSIT": "PickLocationEntity.IntransitQty"  
},  
"parentNode": "WorkExecutionChildEntities",  
"SkipCount": 0  
}
```

SRC

```
SkipCount: 0  
ToLoc: "SHP-1"  
TransportContId: null  
WorkExecutionChildEntities: [{,}, {,-}]  
  0: {,-}  
  1: {,-}  
    Account: "001"  
    Company: null  
    ContainerID: null  
    ContainerType: null  
    ConvertedQty: 3  
    ConvertedQtyUM: "EA"  
    CycleCount: null  
    EndDateTime: "0001-01-01T00:00:00"  
    EquipmentLoc: null  
    FromCheckDig: null  
    FromLoc: "100." WorkExecutionChildEntities[1].IncomingPDLoc  
    FromLocInvAttributesId: 0  
    FromLocationWorkZone: null  
    FromQty: 3  
    FromTrackContainers: "Y"  
    FromVerificationMeth: "None"  
    FromWhs: "WH"  
    InCheckDig: null  
    InTrackContainers: null  
    InVerificationMeth: null  
    InWorkZone: null  
    IncomingPDLoc: null  
    InternalContainerNum: 0
```

DataModel



If the parentNode is not specified, then the SAVE engine will directly map the value in the information node to the DataModel.

Example

WAREHOUSE MOBILE GUIDE

```
"information": {
  "QUANTITY": "Quantity",
  "UM": "QuantityUM",
  "CONVERTEDQUANTITY": "ConvertedQty",
  "CONVERTEDQUANTITYUM": "ConvertedQtyUM"
},
"placeholder": "Enter your Qty",
```

SRC

```
ContainerId: null
ConvertedQty: 15
ConvertedQtyUM: "EA"
CurrentLocation: "F"
DisableQuantityLocationInfo: false
EnteredQuantity: 15
FromCheckDig: null
FromLoc: "100.16"
Full: false
GroupPickOrPutaway: true
IsPartial: null
IsShortPicked: null
IsShortPutaway: null
IsSplitShippingContainer: null
Item: "NOR0005"
ItemDesc: "Puffs Family Size Basic Facial Tissue (EA dims or
LaunchNum: 0
LogisticsUnit: "RSPLP2"
Lot: null
LotExpirationDate: null
OverrideLocation: null
OverrideLot: null
ParentInternalContainerNum: 0
ProcessStartDatetime: "2019-12-06T11:03:48.5139236Z"
Quantity: 15
QuantityUM: "EA"
```

DataModel

Quantity	UM
15	EA
Converted quantity	UM
15	EA

CLOSE

DateTime Control type

Datetime control type is a control which provides flexibility to scan/enter date.

Inventory adjustment

Adjustment type

Positive Adjustment

Location①

100.11

Item①

LOT0013

Company

02-Fred's Food Company

Description

30 ct. Halls Cough Drops Honey Lemon

Quantity①

2

UM

EA

Status

Available

Lot

LOT1212

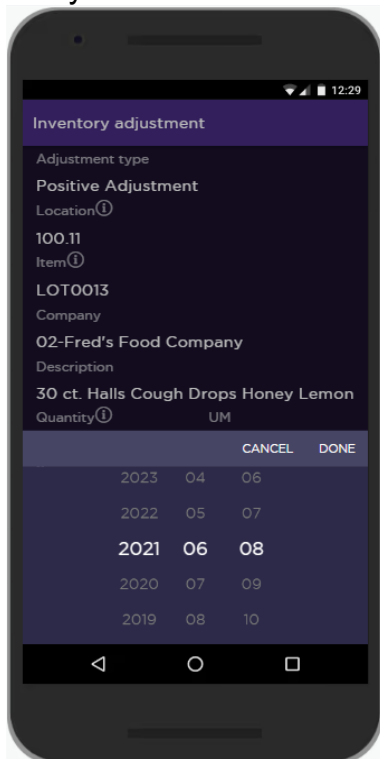
Expiration date

YYYY-MM-DD

GO

WAREHOUSE MOBILE GUIDE

When the user clicks on the DateTime field a picker will come to select the date, time, and year.



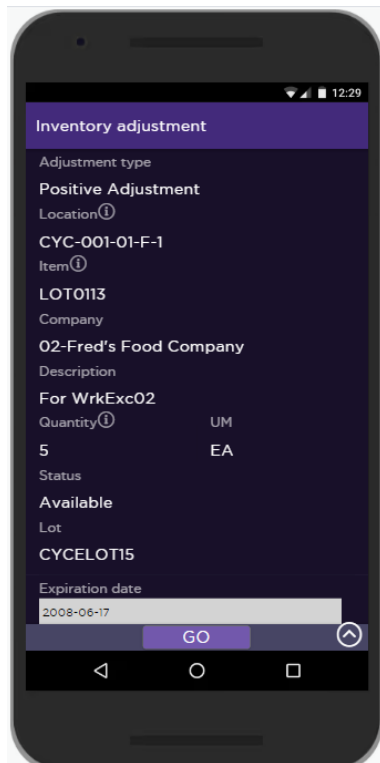
Datetime picker provides two buttons, Cancel and Done. Cancel will close the picker and Done will set the value to the control/widget.

Rules to be followed to add a datetime control to SRC: -

- There can be a maximum of one datetime control per page in SRC.
- The control should contain "displayFormat" which will help in displaying the exact format of field's value on the screen. Example - MM/DD/YYYY, YYYY-MM-DD, etc.

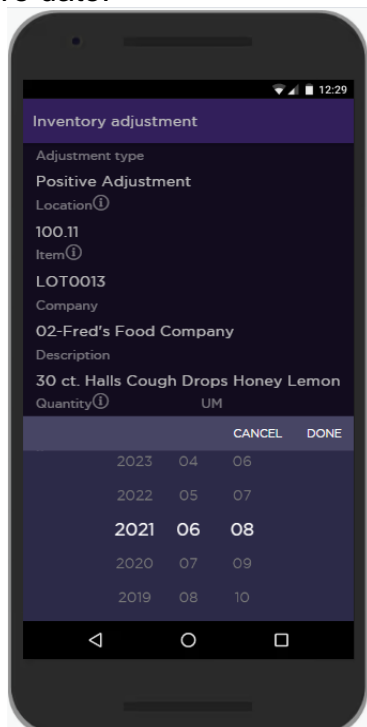
```
{  
  "controltype": "datetime",  
  "label": "EXPIRATIONDATE",  
  "type": "text",  
  "name": "ExpirationDate",  
  "displayFormat": "YYYY-MM-DD",  
  "ShowOnAllPage": "false",  
  "visible": "true"  
}
```

WAREHOUSE MOBILE GUIDE



- The format of the date picker will be the same as we define displayFormat in SRC.

Example - in YYYY-MM-DD we have year first then we have the month and then we have date.



WAREHOUSE MOBILE GUIDE

Custom Date formatting for Labels

From database any date delivered by API call with DataModel will be in ISO format: "YYYY-MM-DD"

Any date (ex: ExpireDate) that will be sent to server must be in ISO format ("YYYY-MM-DD"). DatePicker presented above will automatically format the entered date in ISO before sending the date as input to any API call.

Old format "MMDDYYYY" is also accepted as API date input value.

Any Label date content can be formatted when we add in SRC to Label control the attribute "displayFormat".

- If "displayFormat" attribute is filled with a valid format, then Warehouse Mobile app will convert the content of label in specified date format.

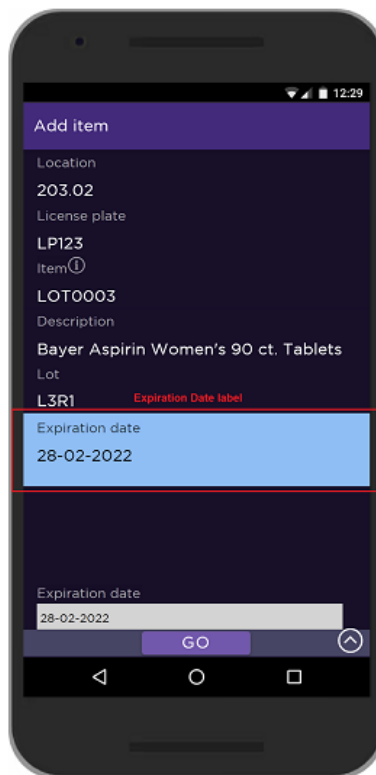
Example: displayFormat filled with "DD-MM-YYYY" (Day Month Year in digits)

```
{
  "controltype": "label",
  "label": "EXPIRATIONDATE",
  "type": "date",
  "visible": "@ShowExpDate",
  "name": "ExpirationDate",
  "displayFormat": "DD-MM-YYYY",
  "ShowOnAllPage": "true",
  "order": 6
}
```

SRC

```
AffectedContainersCount: 0
AffectedLocationsCount: 0
Company: null
ExistingExpirationDate: "2022-02-28"
ExistingInventoryStatus: "Available"
ExpirationDate: "2022-02-28"
FromLoc: "203.02"
FrozenLotInventoryStatus: "Unavailable"
FrozenStatusConfirmationMessage: "Due to the inve
InventoryStatus: [{Identifier: "Available", Descr
IsLotControlled: true
IsSystemDirectedWork: false
Item: "LOT0003"
ItemDesc: "Bayer Aspirin Women's 90 ct. Tablets"
ItemImageUrl: null
LocInvAttributesId: 0
LogisticsUnit: "LP123"
Lot: "L3R1"
```

DataModel



When defining the **displayFormat** attribute with time:

- If you specify a format without an AM/PM indicator (e.g., "MM/DD/YYYY hh:mm:ss"), the time will be displayed in **24-hour format**.

Example: 09/10/2025 18:13:24

- If you include an AM/PM indicator (e.g., "MM/DD/YYYY hh:mm:ss a" or "MM/DD/YYYY hh:mm:ss ampm"), the time will be displayed in **12-hour format**.

WAREHOUSE MOBILE GUIDE

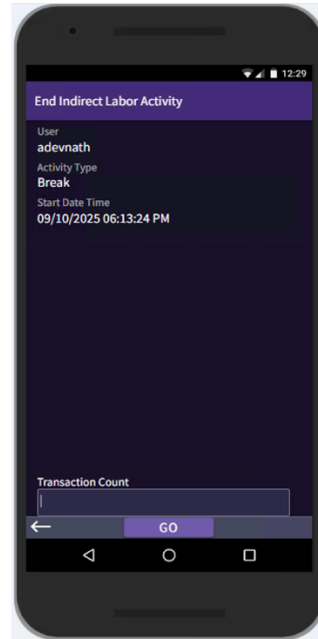
Example: 09/10/2025 06:13:24 PM

```
{
  "controltype": "label",
  "label": "STARTDATETIME",
  "type": "date",
  "name": "ActivityStartDateTime",
  "displayFormat": "MM/DD/YYYY hh:mm:ss a",
  "ShowOnAllPage": "true",
  "order": 3
},
```

SRC

```
"ConfirmationMessage": null,
"Selected_ActivityTypes": "Break",
"ActivityStartDateTime": "2025-09-10T12:43:24.3004295Z",
"ActivityEndDateTime": "0001-01-01T00:00:00",
"ProcessType": 19,
"TransactionCount": 0,
"userdef1": null,
"userdef2": null,
"userdef3": null,
"userdef4": null,
"userdef5": null,
```

DataModel



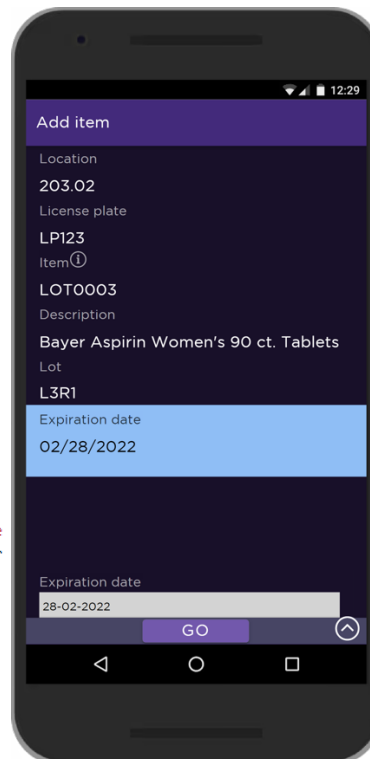
- To instruct the application to format the ExpirationDate label data to the device browser culture language we add the "displayFormat" attribute in SRC with an empty value:

```
{
  "controltype": "label",
  "label": "EXPIRATIONDATE",
  "type": "date",
  "visible": "@ShowExpDate",
  "name": "ExpirationDate",
  "displayFormat": "",
  "ShowOnAllPage": "true",
  "order": 6
}
```

SRC

```
AffectedContainersCount: 0
AffectedLocationsCount: 0
Company: null
ExistingExpirationDate: "2022-02-28"
ExistingInventoryStatus: "Available"
ExpirationDate: "2022-02-28"
FromLoc: "203.02"
FrozenLotInventoryStatus: "Unavailable"
FrozenStatusConfirmationMessage: "Due to the inve
InventoryStatus: [{Identifier: "Available", Descr
IsLotControlled: true
IsSystemDirectedWork: false
Item: "LOT0003"
ItemDesc: "Bayer Aspirin Women's 90 ct. Tablets"
ItemImageUrl: null
LocInvAttributesId: 0
LogisticsUnit: "LP123"
Lot: "L3R1"
```

DataModel



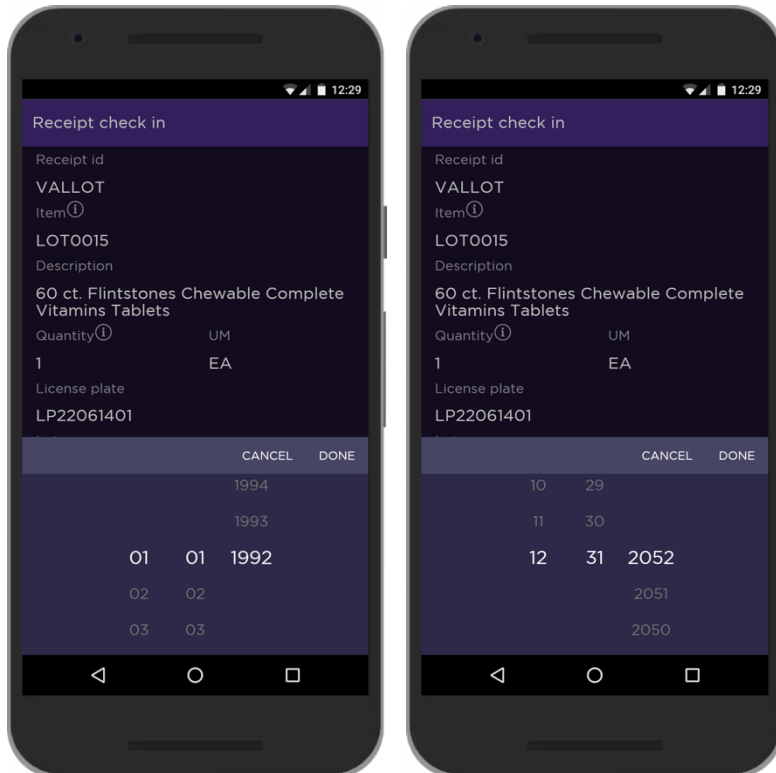
WAREHOUSE MOBILE GUIDE

Note: In SRC Do not add "displayFormat" attribute to controls that do not represent a date (ex: LicencePlate). If added Mobile will try to convert the control value taken from DataModel to a date and if succeeds displayed data will be inaccurate.

Minimum and Maximum values for the DateTime control

The possible values for selection in the DateTime control will be between -30 years from the current year and +30 years from the current year.

Ex. Today is 2022-06-17 then the possible selectable values will range from 1992-01-01 and 2052-12-31



Custom Widget Controls

We have introduced new control types to the SRC, they are

- lpEntry
- snEntry
- lotEntry

LPEntry Control type

Lpentry control type is a custom control type (widget) which provides flexibility to putaway the inventory to multiple license plates.

To show lpentry screen to the user, one must include a dedicated page for LPEntry in the SRC file and the SRC file must adhere to the following guidelines

- There should be two main controls with the names "LogisticsUnit" and "Quantity" with controltype "lpentry"
- Both the controls mentioned above should be the first two controls of the page

WAREHOUSE MOBILE GUIDE

- The page can have any number of label controls.
- The page should not have any textbox controls as the controltype "lpentry" will internally act as textbox

Below is the example of SRC having "lpEntry" control:

```
"Controls": [  
  {  
    "controltype": "lpEntry",  
    "label": "LICENSEPLATE",  
    "type": "text",  
    "name": "LogisticsUnit",  
    "validations": {  
      "required": true,  
      "asyncValidator": {  
        "uri": "/general/scaleapi/LocationInventoryApi/ValidateLicensePlate?licensePlate=",  
        "model": {  
          "FromContainerId": "containerId",  
          "ToLocation": "ToLocation"  
        }  
      }  
    },  
    "pattern": "",  
    "minLength": "",  
    "maxLength": "20"  
  },  
  {  
    "visible": "true",  
    "ShowOnAllPage": "true",  
    "order": 4  
  }  
],  
{  
  "controltype": "lpEntry",  
  "label": "PICKQTY",  
  "type": "number",  
  "name": "Quantity",  
  "ShowOnAllPage": "true",  
  "visible": "true",  
  "validations": {  
    "required": true,  
    "pattern": "",  
    "minLength": "",  
    "maxLength": "20",  
    "rangeStart": 0.000001,  
    "rangeEnd": "@Quantity"  
  }  
},
```

WAREHOUSE MOBILE GUIDE

SnEntry Control Type

SnEntry control type is a custom control type (widget) which provides flexibility to scan/enter serial numbers.

To show snEntry screen to user, one must include either a dedicated page for SnEntry or a separate screen flow in the SRC file and the SRC file must adhere to the following guidelines:

- There should be one main control with name "SerialNumber" with controltype "snEntry"
- The above-mentioned control should be the first control of the page.
- The page can have any number of label controls.
- The first label(second json object on the page) must represent the Quantity that was entered or the count of serial numbers that get scanned
- The page should not have any textbox controls as the controltype "snEntry" will internally act as textbox

Below is the example of SRC having "snEntry" control during receipt check-in:

```
"Controls": [  
  {  
    "controltype": "snEntry",  
    "label": "SERIALNUMBER",  
    "type": "text",  
    "name": "SerialNumber",  
    "validations": {  
      "required": true  
    },  
    "visible": "true",  
    "ShowOnAllPage": "true",  
    "order": 1  
  },  
  {  
    "controltype": "label",  
    "label": "ONHANDQUANTITY",  
    "type": "text",  
    "width": 50,  
    "name": "EnteredOnHandQuantity",  
    "placeholder": null,  
    "required": false,  
    "ShowOnAllPage": "true",  
    "order": 9  
  },  
  {  
    "controltype": "label",  
    "label": "UM",  
    "name": "Selected_UnitOfMeasures",  
    "type": "text",  
    "width": 50,  
    "order": 10,  
    "ShowOnAllPage": "true"  
  },  
]
```

WAREHOUSE MOBILE GUIDE

lotEntry Control Type

lotEntry control type is a custom control type (widget) that provides flexibility to scan/enter lot and lot attributes when an Item has a lot template defined.

To show **lotEntry** screen to the user, one must include either a dedicated page for **lotEntry** or a separate screen flow in the SRC file and the SRC file must adhere to the following guidelines:

- There should be one main control with the name "Lot" with controltype "**lotEntry**"
- The above-mentioned control should be the **first control** of the page.
- The page can have any number of label controls.
- The page should not have any textbox controls as the controltype "**lotEntry**" will internally act as textbox.

Below is the example of SRC having "lotEntry" control during receipt check-in:

```
Receiving.json  X
Schema: <No Schema Selected>
698         "visible": "@DisplayContainerCountDetails",
699         "ShowOnAllPage": "true"
700     },
701     {
702         "controltype": "label",
703         "label": "LICENSEPLATE",
704         "type": "text",
705         "name": "LicensePlate",
706         "ShowOnAllPage": "true",
707         "order": 9
708     },
709     {
710         "label": "LOT",
711         "controltype": "lotEntry",
712         "type": "text",
713         "name": "Lot",
714         "ShowOnAllPage": true,
715         "visible": "true",
716         "validations": {
717             "required": true,
718             "pattern": "",
719             "asyncValidator": {
720                 "uri": "/inventory/scaleapi/LotApi/Validate-Lot-And-Template?lotOrAttributeValue=",
721                 "model": {
722                     "Company": "Company",
723                     "Item": "Item",
724                     "Lot": "Lot",
725                     "LotAttributeEntities": "LotAttributes",
726                     "Warehouse": "Warehouse",
727                     "ExpirationDate": "ExpirationDate"
728                 }
729             }
730         }
731     }
732 ]
733 }
734 }
735 }
```

WAREHOUSE MOBILE GUIDE

Picker Control Type

Picker control type is a custom control type (widget) which provides flexibility to pick or scan a value from an array with two fields, Identifier and Description.

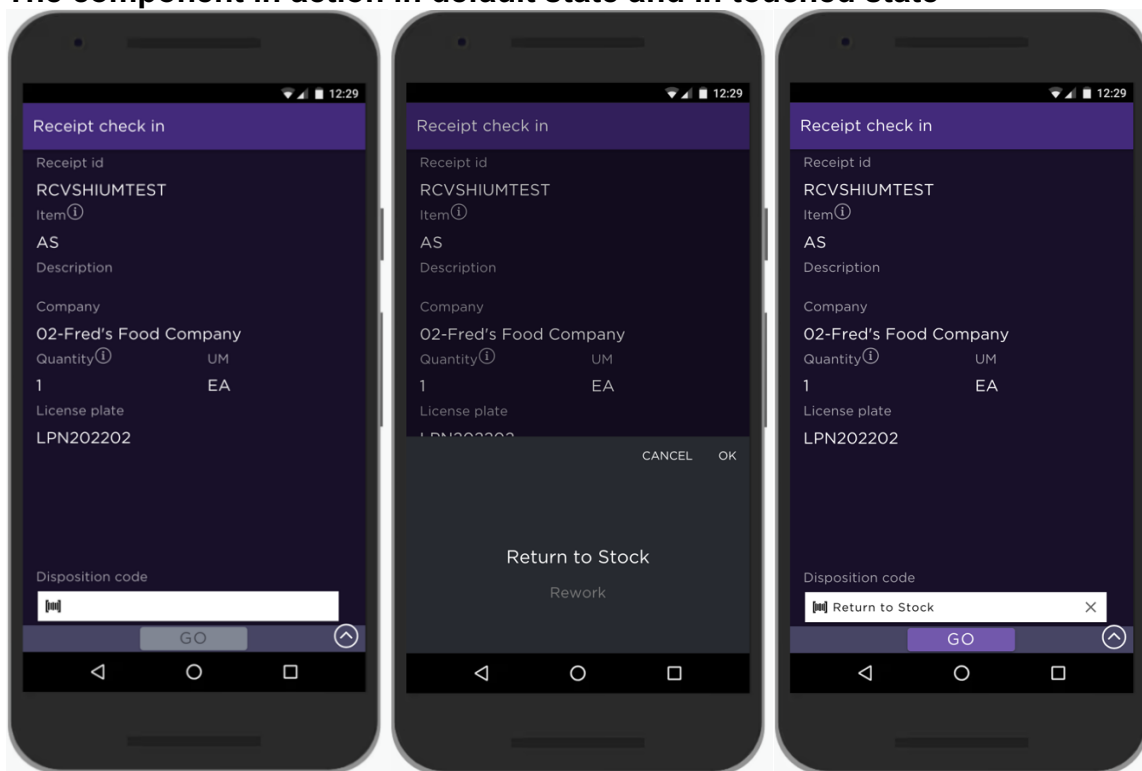
The SRC configuration to activate the picker control

```
{
  "controltype": "picker",
  "label": "DISPOSITIONCODE",
  "type": "text",
  "name": "DispositionCodes",
  "visible": "@RequireDispositionCode",
  "validations": {
    "required": true
  },
  "ShowOnAllPage": "true",
  "order": 15
}
```

The DataModel from the API payload

```
▼ DispositionCodes: [{Identifier: "A", Description: "Return to Stock"}, {Identifier: "B", Description: "Rework"}]
  ▼ 0: {Identifier: "A", Description: "Return to Stock"}
    Description: "Return to Stock"
    Identifier: "A"
  ▼ 1: {Identifier: "B", Description: "Rework"}
    Description: "Rework"
    Identifier: "B"
```

The component in action in default state and in touched state



DataModel update

After the value selection the DataModel will be updated with the following fields

Selected_*[original field name]*

WAREHOUSE MOBILE GUIDE

SelectedDescription_*[original field name]*

For our sample with disposition code, where the original field name was **DispositionCodes** the updated DataModel will have:

```
SelectedDescription_DispositionCodes: "Return to Stock"
```

```
Selected_DispositionCodes: "A"
```

Additional picker configuration to switch from Description to Identifier

In the SRC file on the picker control configuration, there is a new property introduced “**textColumn**” where we can specify which column from the bound data model array to be displayed in the picker control.

In our example, we set up a picker control to display the container types, and we would like to show on the user interface instead of the **Description** (which is the default) the **Identifier** column.

The SRC configuration to activate the picker control with the Identifier

```
{
  "controltype": "picker",
  "label": "CONTAINERTYPE",
  "type": "text",
  "name": "ContainerTypes",
  "validations": {
    "required": true
  },
  "ShowOnAllPage": "true",
  "visible": "true",
  "order": 3,
  "textColumn": "Identifier"
}
```

To activate with the Description column, either we miss completely the property “**textColumn**” as by default the picker will display the Description or we specify explicitly the Description column

```
{
  "controltype": "picker",
  "label": "CONTAINERTYPE",
  "type": "text",
  "name": "ContainerTypes",
  "validations": {
    "required": true
  },
  "ShowOnAllPage": "true",
  "visible": "true",
  "order": 3
}
```

or with the same effect

```
{
  "controltype": "picker",
  "label": "CONTAINERTYPE",
  "type": "text",
  "name": "ContainerTypes",
  "validations": {
    "required": true
  },
  "ShowOnAllPage": "true",
  "visible": "true",
  "order": 3,
  "textColumn": "Description"
}
```

WAREHOUSE MOBILE GUIDE

Additional picker configuration to enable scanning a location that is not in the original list for override pick

While performing an override pick, if the returned location list does not contain the wanted location for override you can scan a new valid location. In the SRC there needs to be a property for the location verification API. The required property is:

"ValidateScannedUri":"/outbound/scaleapi/WorkInstructionsApi/ValidateScannedOverrideLocation"

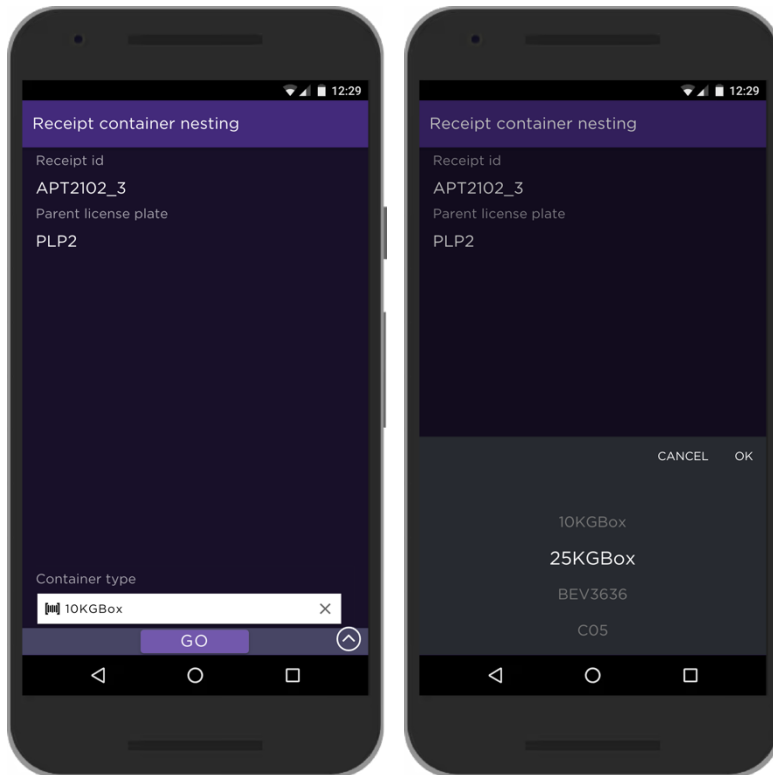
```
{
  "controltype": "picker",
  "label": "LOCATION",
  "type": "text",
  "name": "LocationInventoryList",
  "ShowOnAllPage": "true",
  "order": 4,
  "visible": "true"
  "ValidateScannedUri":"/outbound/scaleapi/WorkInstructionsApi/ValidateScannedOverrideLocation"
}
```

The DataModel from the API payload

```
▼ ContainerTypes: [{Identifier: "10KGBBox", Description: "10 Kilogram Box (Carrier specific)"},...]
  ► 0: {Identifier: "10KGBBox", Description: "10 Kilogram Box (Carrier specific)"}
  ► 1: {Identifier: "25KGBBox", Description: "25 Kilogram Box (Carrier specific)"}
  ► 2: {Identifier: "BEV3636", Description: "36 x 36 Plastic Beverage Pallet"}
  ► 3: {Identifier: "C05", Description: "Carton 12 x 8 x 4"}
  ► 4: {Identifier: "C08-Tall", Description: "Carton 12 x 6 x 12"}
  ► 5: {Identifier: "C10", Description: "Carton 16 x 8 x 8"}
  ► 6: {Identifier: "C18", Description: "Carton 16 x 16 x 12"}
  ► 7: {Identifier: "C20", Description: "Carton 24 x 24 x 16"}
  ► 8: {Identifier: "CHEP4840", Description: "48 x 40 CHEP Pallet"}
  ► 9: {Identifier: "CLG", Description: "Carton - Large"}
  ► 10: {Identifier: "CS", Description: "Full Case"}
  ► 11: {Identifier: "CTVali", Description: "Carton 12 x 8 x 4"}
  ► 12: {Identifier: "EA", Description: "Each"}
  ► 13: {Identifier: "GMA4840", Description: "48 x 40 GMA Pallet"}
  ► 14: {Identifier: "L20", Description: "Extra Long Carton"}
  ► 15: {Identifier: "L30", Description: "Extra Long Carton - Smaller Size"}
  ► 16: {Identifier: "OS1", Description: "Oversized - Level 1"}
  ► 17: {Identifier: "OS2", Description: "Oversized - Level 2"}
  ► 18: {Identifier: "P4248", Description: "Pallet 42 x 48"}
  ► 19: {Identifier: "PalletIC", Description: "Pallet - Infinite Capacity (Volume and Weight)"}
  ► 20: {Identifier: "PL", Description: "Full Pallet"}
```

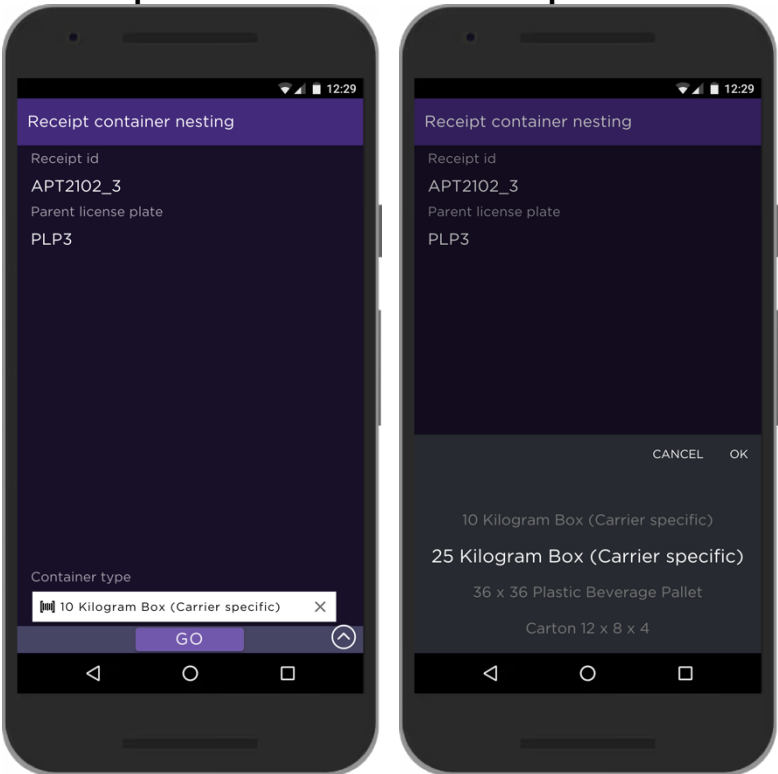
WAREHOUSE MOBILE GUIDE

The component switched to “Identifier” in default state and in touched state



WAREHOUSE MOBILE GUIDE

The component switched to “Description” in default state and in touched state



DataModel update

After the value selection the DataModel will be updated with the following fields no matter what is the “textColumn” property

- Selected_[original field name]** – will always get the Identifier value
- SelectedDescription_[original field name]** – will always get the Description value

For our sample with container types, where the original field name was **ContainerTypes** the updated DataModel will have:

```
SelectedDescription_ContainerTypes: "25 Kilogram Box (Carrier specific)"
Selected_ContainerTypes: "25KGBBox"
```

Slider control type

A slider control type can be used when you wish to present multiple options for the user to select, and detailed information about the options needs to be presented. The slider will be presented as a modal in front of the existing screen and the user will be able to slide left-right to see all the options and Select an option using a button on each slide. Ensure that it is the only control on its page and the only page in its screen flow. Below are the elements in this control type:

Element	Use
controlType	selectSlides

WAREHOUSE MOBILE GUIDE

information	All fields to be displayed in each slide should be mentioned in this element with correct property name from the API response.
Label	Title to be displayed for the slider
parentNode	The array element in the API response which contains the objects for this slider
name	Can be anything, just a unique name for this control
visible	True / false

This is an example of a select slide element from an SRC:

```
{
  "controltype": "selectSlides",
  "information": {
    "ERPLINENUMBER": "ERPLineNumber",
    "ITEM": "Item",
    "COMPANY": "Company",
    "DESCRIPTION": "ItemDesc",
    "LOT": "Lot",
    "QUANTITY": "Quantity",
    "UM": "QuantityUm",
    "ImageUrl": "ItemImageUrl"
  },
  "Label": "SELECTARECEIPTLINE",
  "parentNode": "CheckinContainerEntities",
  "name": "CheckinContainerEntities",
  "visible": "true",
  "order": 1
}
```

Below is screenshot for a slide for above SRC example:

WAREHOUSE MOBILE GUIDE

Receipt initiation

1 / 2

ERP line number
1.00000

Item
NOR-0001

Company
02-Fred's Food Company

Description
Tide - 12 Oz.

Quantity	UM
2	EA



GO

Special link between Slider Control and Picker Control

In the special case that there is a Slider Control and a Picker Control on the same page, and both controls are bound to the same array in the data model, the WarehouseMobile platform will reflect changes in both of them, respectively if we slide with the slider to an item, also in the picker will be selected the same item, and vice versa if we choose in the picker an item, the slider will slide to the corresponding item as well. In case of scanning an identifier value from the picker, both controls will select the item scanned.

The SRC configuration to link the Slider control and Picker control

WAREHOUSE MOBILE GUIDE

```
"Controls": [  
  {  
    "controltype": "selectSlides",  
    "information": {  
      "LOCATION": "Identifier",  
      "ITEM": "Item",  
      "DESCRIPTION": "ItemDescription",  
      "LOT": "Lot",  
      "EXPIRATIONDATE": "ExpirationDate",  
      "QUANTITY": "Quantity",  
      "UM": "Um",  
      "WORKZONE": "WorkZone"  
    },  
    "Label": "LOCATION",  
    "parentNode": "LocationInventoryList",  
    "name": "LocationInventoryList",  
    "visible": "true",  
    "order": 3  
  },  
  {  
    "controltype": "picker",  
    "label": "LOCATION",  
    "type": "text",  
    "name": "LocationInventoryList",  
    "ShowOnAllPage": "true",  
    "order": 4,  
    "visible": "true"  
  }  
]
```

Note:

The fields label ending in “DATE” will be displayed in local date culture

This applies also to Information modal box fields as well as for Slider control fields

In case that the field from payload does not end in “DATE” still we can adjust the suffix of the label resource, and date culture will be applied.

The DataModel from the API payload

```
▼ LocationInventoryList: [{Identifier: "001.15.2", Description: "001.15.2 (10 EA)", Location: "001.15.2", Item: "LOT0003",...},...]  
  ▼ 0: {Identifier: "001.15.2", Description: "001.15.2 (10 EA)", Location: "001.15.2", Item: "LOT0003",...}  
    AllocationZone: "W3ARackReserve"  
    CheckDig: "2"  
    Description: "001.15.2 (10 EA)"  
    ExpirationDate: "2022-05-12"  
    Identifier: "001.15.2"  
    Item: "LOT0003"  
    ItemDescription: "Bayer Aspirin Women's 90 ct. Tablets"  
    Location: "001.15.2"  
    Lot: "L3"  
    Quantity: 10  
    Um: "EA"  
    WorkZone: "W3WRackUpper"  
  ▼ 1: {Identifier: "001.11.2", Description: "001.11.2 (44 EA)", Location: "001.11.2", Item: "LOT0003",...}  
    AllocationZone: "W3ARackReserve"  
    CheckDig: "2"  
    Description: "001.11.2 (44 EA)"  
    ExpirationDate: "2022-01-06"  
    Identifier: "001.11.2"  
    Item: "LOT0003"  
    ItemDescription: "Bayer Aspirin Women's 90 ct. Tablets"  
    Location: "001.11.2"  
    Lot: "L3"  
    Quantity: 44  
    Um: "EA"  
    WorkZone: "W3WRackUpper"
```

Screenshot Slider linked with Picker (actual override pick location selection)

WAREHOUSE MOBILE GUIDE

Override location

1 / 2

Location
001.15.2

Item
LOT0003

Description
Bayer Aspirin Women's 90 ct. Tablets

Lot
L3

Expiration date
05/12/2022

Quantity
10

UM
EA

Work zone
W3WRackUpper

Location
001.15.2

GO

Camera Control type

The camera control type is a control that helps to take pictures with the device camera, this control will be visible only if camera hardware is present on the device.

WAREHOUSE MOBILE GUIDE

Receipt check in

Receipt id
TEST_1_22_24

Item ⓘ
NOR0005

Description
Puffs Family Size Basic Facial Tissue (EA dims only)

Quantity ⓘ	UM
1	EA

License plate
LP

GO

When the user clicks on the camera icon default back camera will be opened, once the user takes the photo a preview screen will appear where the user will have to retake, select, and close options.

- Select the option to selects taken photo and take the user to the next screen.
- The retake option clears the existing photo and opens the camera again.
- Close will close camera

WAREHOUSE MOBILE GUIDE



Below is the example of SRC having “Camera” control:

WAREHOUSE MOBILE GUIDE

```
"Actions": [
  {
    "Type": "Back",
    "Enable": "true"
  },
  {
    "Type": "Camera",
    "Enable": "true",
    "Label": "SAVEIMAGE",
    "CameraMappingModel":
    [
      { "name": "ReferenceId", "dependencyname": "ReferenceTypes" },
      { "name": "Company" }
    ],
    "InitialModelURI": "/general/scaleapi/DocumentManagementApi/ImageInitialModel?category=RECEIVING",
    "Dialog": {
      "Controls": [
        {
          "controltype": "dropdown",
          "label": "REFERENCETYPE",
          "name": "ReferenceTypes",
          "validations": {
            "required": true
          },
          "ShowOnAllPage": "true",
          "order": 1
        },
        {
          "controltype": "textbox",
          "label": "REFERENCEID",
          "type": "text",
          "name": "ReferenceId",
          "validations": {
            "required": true
          },
          "ShowOnAllPage": "true",
          "order": 2
        },
        {
          "controltype": "textarea",
          "label": "NOTES",
          "name": "Notes",
          "visible": "true",
          "order": 3,
          "validations": {
            "maxLength": "@FieldLengthNote"
          }
        }
      ]
    }
  },
  {
    "uri": "/general/scaleapi/DocumentManagementApi/SaveImage"
  }
],
```

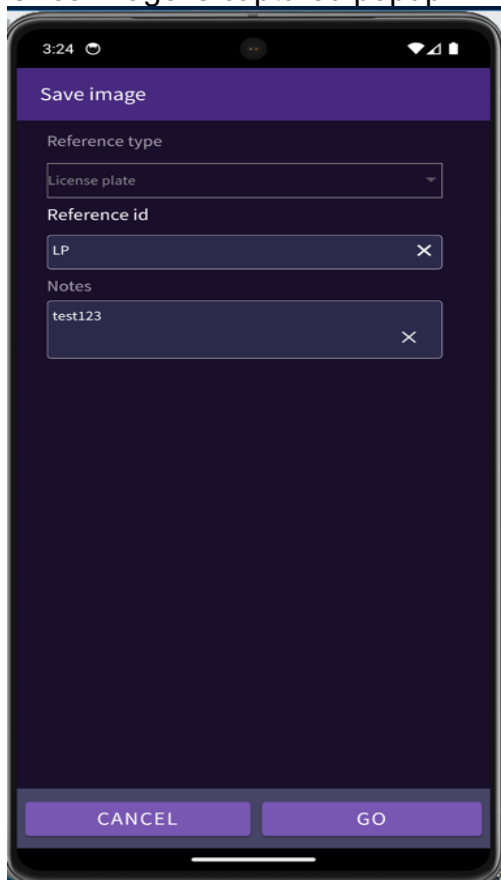
Attributes

Attribute	Type	Description
-----------	------	-------------

WAREHOUSE MOBILE GUIDE

Type	camera	This will enable camera icon in title bar
Label	string	This will be title of the popup and it should always be a resource key.
Enable	Boolean	This flag will determine the visibility of the camera icon on each page level
CameraMappingModel	Object	This object indicates interrelated properties that need to be updated from the parent mapping model.
InitialModelURI	string	URI which will fetch the required model for camera component.
Dailog	Object	Which is a collection DOM controls which will use to capture details of the image after taking an image.
URI	string	Endpoint where the camera model will be posted

Once image is captured popup will be appeared, to get image details as shown below.



3:24

Save image

Reference type

License plate

Reference id

LP

Notes

test123

CANCEL GO

WAREHOUSE MOBILE GUIDE

Textarea Control type

This control will render textarea on DOM which will collect user inputs like comments or reviews.

Below is sample of SRC, having text area control.

```
{
  "controltype": "textarea",
  "label": "NOTES",
  "name": "Notes",
  "visible": "true",
  "order": 3,
  "validations": {
    "maxLength": "@FieldLengthNote"
  }
}
```

Attribute	Type	Description
controltype	string	A controltype type defines the DOM element to be shown on the browser or the device (textarea,textbox, label, dropdown etc.,).
label	string	Name of the control that is displayed and it should always be a resource key.
name	string	An important attribute required to map the control with the DataModel coming from the server.
order	int	Defines the order number of a control in a page.
validations	object	Contains different types of client-side validations on a DOM element.
visible	string	Makes the control visible on the screen if it is set to "true".

WAREHOUSE MOBILE GUIDE

DataGrid Control Type

DataGrid control type is a custom control type which provides flexible, tabular interface for displaying and interacting with data organized into rows and columns. The control is reusable and responsive with features like row selection, highlighting inactive rows, displaying row counts, horizontal scrolling and vertical scrolling.

View Picks

Pick Instructions (17)

Seq	Item	Loc	Qty
10	NOR-0020	101.020.1	5
11	NOR-0021	101.020.2	2
12	NOR-0022	101.020.3	9
13	NOR-0023	101.020.4	9
14	NOR-0023	101.020.4	9
15	NOR-0024	101.021.2	2
0	NOR-0010	101.070.2	2
1	NOR-0011	101.070.1	2
2	NOR-0012	101.018.1	2

Sequence

CANCEL

GO

WAREHOUSE MOBILE GUIDE

The SRC configuration to activate the datagrid control

```
{
  "controltype": "datagrid",
  "label": "PICKINSTRUCTIONS",
  "name": "PickInstructions",
  "selectRow": "true",
  "data": {
    "uri": "/outbound/scaleapi/WorkInstructionsApi/pickInstructions",
    "params": {
      "workUnit": "WorkUnit",
      "workProfile": "WorkProfileName"
    }
  },
  "keyColumn": "SEQ",
  "ActionModelData": {
    "IsViewPicks": true,
  },
  "UpdateControl": {
    "EnteredSequence": "Selected_PickInstructions.SEQ"
  },
  "inactiveColumn": "HIDDEN_PICKED"
},
```

Attribute	Type	Description
controltype	string	A controltype type defines the DOM element to be shown on the browser or the device (textbox, label, dropdown etc.).
label	string	Name of the control that is displayed and it should always be a resource key.
name	string	An important attribute that is required to map the control with the DataModel coming from the server.
selectRow	boolean	Determines whether a user can select an entire row
data	object	Contains url and params to make an external call to fetch data for the grid If the data property is not specified, then it takes data from datamodel with the property named same as control name
keyColumn	string	Defines the unique column for the grid. If this property is not specified then row selection will not work
updateControl	object	It is an optional control attribute used to specify the control name that needs to be updated upon row selection
inactiveColumn	string	It is an optional control attribute that is used to specify the column name used to highlight the inactive row.

WAREHOUSE MOBILE GUIDE

		The columns should contain true/false or 0/1 as a values
--	--	--

The DataModel from the API payload

```
PendingWorkUnit: null  
PickInstructions: []  
PreviousInstructionQty: 8
```

The datamodel property name should match with name of the control. PickInstruction should be an empty array or array of instruction objects

DataModel update

After the value selection the DataModel will be updated with the following fields

Selected_*[original field name]*

Custom redirection of the execution flow

Whenever the warehouse mobile application requests to the SCALE API's will receive a payload, containing the fields needed for the screen or execution flow, if in the response the SRCFlow property is completed, the warehouse mobile platform will open the corresponding flow on the device.

SCALE has powerful extensibility capabilities, one of which is to override WEB API calls and add custom data to the payload that is arriving to the Warehouse Mobile. This allows us to bind these custom fields to the SRC Flow controls, and labels.

All fields added by an exit point will be added to the custom property of the payload.

These fields must be bound by the "custom." Prefix in the control name field.

Now we have a new possibility to add also the property "SRCFlow" into the custom node from the exit point. In this case, the WarehouseMobile platform will notice that this field is completed and if the specified flow exists in the currently loaded SRC file, the execution will be redirected to that flow. This is a powerful feature as will enable PSO or customer to completely new flows to the execution.

Ex. Extending the current flow with a new step, checking for truck status for receiving:

WAREHOUSE MOBILE GUIDE



Sample payload

```
{
  ....
  "custom" : {
    "SRCFlow" : "CustomEntry",
    "TruckStatuses": [
      {"Identifier": "CLEAN", "Description": "Truck is clean"},
      {"Identifier": "DIRTY", "Description": "Truck is dirty"}
    ]
  }
}
```

Warning Message

A new type of toast message that can be used to present relevant information to the user.

WAREHOUSE MOBILE GUIDE

As an example this is used on Location Lookup Screen for override pick to direct the user to scan a location when the picker control is not pre-populated with locations.

The DataModel from the API payload

```
▼ WarningMessage: {WarningCode: "MSG_OVERRIDEPICK41",...}
  Message: "No locations with available quantity found. Enter a location to swap with allocated quantity."
  ToastDuration: 10
  WarningCode: "MSG_OVERRIDEPICK41"
```

To have a warning message be shown to the user, the API must respond with “WarningMessage” property, the ToastDuration property can be used to specify the duration of the toast message, if this property is not included Whs Mobile will default to base duration (3 seconds).

Disabling auto-scrolling behavior in Warehouse Mobile

When the warehouse mobile application generates a screen based on the SRC file, it inherently exhibits an auto-scrolling feature. This scrolling action can take the form of automatically moving to the last label on the screen or scrolling down to the highlighted field during the verification process. To override or disable this default behavior, users have the option to include the '**enableAutoScroll**' attribute with a value of 'false'. By setting this parameter to 'false,' the system will refrain from engaging in automatic scrolling behavior for the given SRC rendering, providing users with more control over the display.

For instance, to disable the auto-scrolling for Receiving.json the attribute can be added as shown below:

```
1 {
2   "enableAutoScroll": false,
3   "initialDataModel": {
4     "Model": {
5       "ReceiptId": "",
6       "Item": "",
7       "LicensePlate": "",
8       "ReceivingPreference": "MenuOptionName"
9     }
10  },
11  "flows": [
12    {
13      "flowName": "ReceivingEntry",
14      "currentPage": "",
15      "Title": "RECEIPTINITIATION",
16      "ContinueToNextFlow": true,
17      "Pages": [
18        {
19          "Go": {
20            "uri": "/inbound/scaleapi/ReceiptApi/validate-receipt-header",
21            "nextFlow": "ItemEntry"
22          },
23          "Actions": [
24            {
```

Warehouse Mobile Buffer scanning

Buffer scanning is the method designed to efficiently gather and store barcode scans in sequence into a client-side buffer. This allows easy retrieval of scans within the Warehouse mobile app as required.

WAREHOUSE MOBILE GUIDE

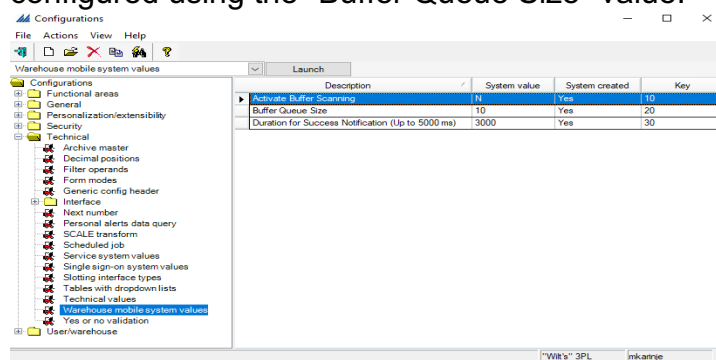
This feature eliminates the need for users to wait for full-screen loading before scanning the next input. Consequently, users can seamlessly perform continuous scans, enabling the warehouse mobile system to promptly capture and transition to the next screen without any delay.

However, it's important to note that in case of system errors, the buffer queue will be cleared. This means that if an error occurs, the user will need to restart scanning from the point of failure.

Warehouse Mobile System Values have introduced two system values that enable users to buffer the scans and manage queue size during Warehouse mobile transactions.

Activate Buffer Scanning

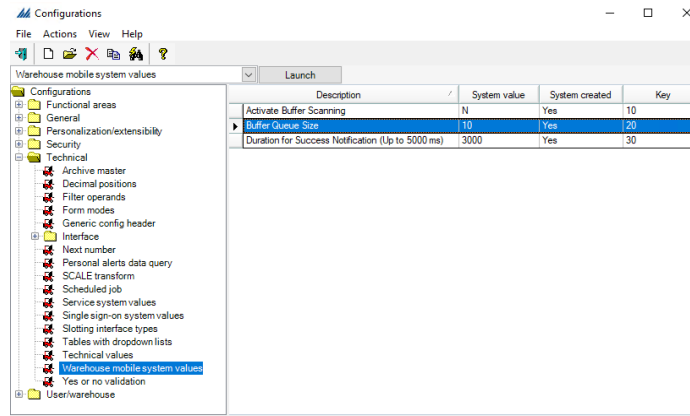
- If set to "Yes," scan buffering is enabled, allowing users to buffer the scans during Warehouse Mobile transactions, with the system maintaining a queue of scanned records.
- The maximum number of scans that can be held in the queue is configured using the "Buffer Queue Size" value.



Buffer Queue Size

- The maximum number of records that can be present in the queue when scan buffering is enabled.
- When the buffer queue size is set to 5, scanning will be disabled when there are a total 5 records in the queue. Once the number of queued records decreases, scanning is re-enabled.
- Additionally, if a user encounters an error during a scan, all unprocessed records in the queue will be cleared, requiring a rescan.

WAREHOUSE MOBILE GUIDE



Warehouse Mobile Progress Bar loader

In warehouse mobile, a new blue indeterminate progress bar appears to indicate any transaction or navigation that involves an API call from the user interface. This progress bar signals that the user interface is waiting for an API response.

The progress bar loader will not appear for API requests that take less than 200 milliseconds.

It is now available under feature flag - `FEATURE_27065_PROGRESS_BAR_LOADER` (by default turned on).

Note that, if feature flag is turned off, then old "Please Wait" loader will appear.

WAREHOUSE MOBILE GUIDE

The screenshot displays the 'Inventory adjustment' screen in the Warehouse Mobile application. The screen has a dark blue header with the title 'Inventory adjustment'. Below the header, there is a list of fields: 'Description', 'Quantity', 'Status', 'Available', 'License plate', 'Lot', 'Expiration date', and three 'User defined field' entries. The 'Description' field contains the text 'Canned Bumble Bee Solid White Tuna in Water'. The 'Quantity' field is set to '2' and has a unit 'UM' next to it. The 'Status' field is set to 'EA'. The 'Available' field is set to 'Available'. The 'License plate' field is set to 'LP900'. The 'Lot' field is set to 'LOT0000'. The 'Expiration date' field is set to '10/12/2025'. The 'User defined field 1' is highlighted with a red box. At the bottom of the screen, there is a 'GO' button and a back arrow icon.

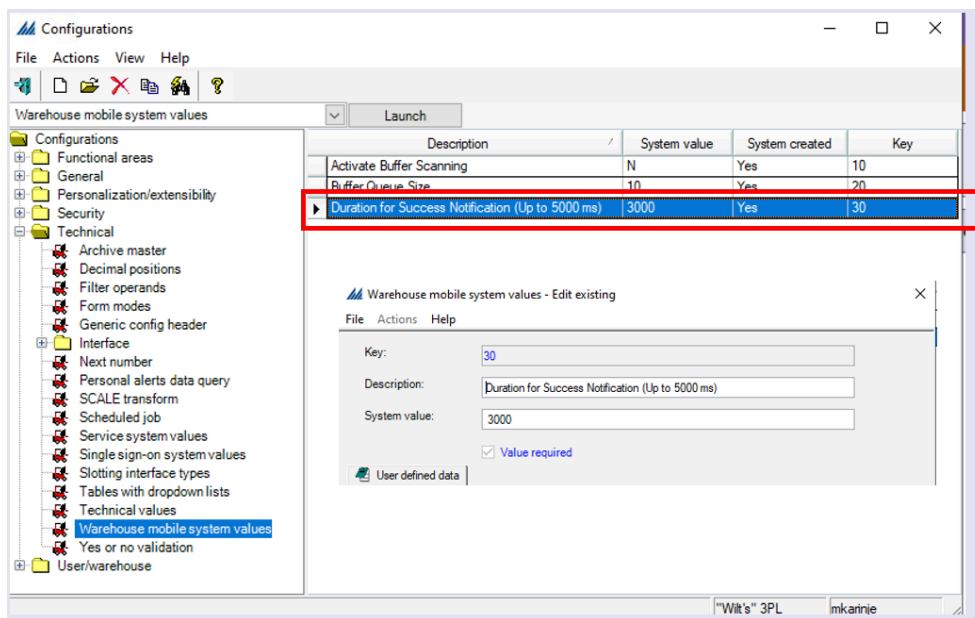
Success Message Duration

In the warehouse mobile application, a new system value has been introduced in our configuration under warehouse mobile system values to control the duration of the success message appearance.

Duration for Success Notification (Up to 5000ms)

- Users can configure a maximum of up to 5000 milliseconds.
- If the user enters a value of more than 5000, for example, 8000, the system will display the success message for 5000 milliseconds only.
- If the value is set to zero, then the success message is not displayed on the screen.

WAREHOUSE MOBILE GUIDE



Warehouse Mobile Multi-Segment Barcode

Multi-Segment Barcode is a feature that enables the Warehouse Mobile application to intelligently parse barcodes containing multiple encoded data fields (such as GS1 barcodes) in a single scan. When a multi-segment barcode is scanned, the application automatically extracts and distributes each encoded field value to the corresponding form control on the screen.

Enabling the Feature

This feature is controlled by the feature flag **FEATURE_3304_MULTI_SEGMENT_BARCODE**. When this flag is enabled, the application loads the supported barcode label types from the environment configured and activates multi-segment barcode parsing during scanning. When the feature flag is disabled, the application falls back to standard single-value barcode scanning behavior.

Configuration

Multi-Segment Barcode requires the following server-side configuration:

WAREHOUSE MOBILE GUIDE

- The feature flag `FEATURE_3304_MULTI_SEGMENT_BARCODE` must be enabled on the SCALE Environment.
- Barcode label types must be configured on the server.

Select a Multi-Segment Label Type

Identifier	Description	System Value 1	Active
☐	GS1	GS1 Label Types	☐
Items per page: 10			

+ ADD NEW RECORD

SCALE CONFIGURATION

Enter Multi-Segment Label Type Information

Identifier	Record Type
GS1	MULTISEGMENTPREFIX
Description	
GS1 Label Types	
System Defined Values	
Prefix	
^(LABEL-TYPE-(DATAMATRIX EAN128 CODE128) \\[dC][0-9])	

The application retrieves the supported label types via the API: `/general/scaleapi/MultiSegmentBarcodeApi/AllBarcodeTypes`. Only barcodes whose label type matches one of the configured patterns will be parsed as multi-segment barcodes.

- The server-side barcode parsing API must be available: `/general/scaleapi/MultiSegmentBarcodeApi/ParseLabel`.

How It Works

When a barcode is scanned on a device that supports label type identification (such as Zebra or Honeywell devices), the following process takes place:

- The scan event is captured along with the barcode label type (e.g., GS1-128, Code128).
- The application checks if the scanned label type matches any of the configured multi-segment barcode label type patterns.

WAREHOUSE MOBILE GUIDE

- If the label type matches, the barcode data is sent to the server-side API (MultiSegmentBarcodeApi/ParseLabel) for parsing. The current DataModel is included in the request payload to provide context.
- The API response returns the parsed fields in a structured format (BarcodeData), with each field containing a Value, an AutoFill flag, and an AutoExecute flag.
- After the barcode is successfully parsed, the resulting BarcodeData structure is stored in saveEntityManager.BarcodeData. This is the central storage location from which all controls and the generic-view-processing pipeline retrieve barcode-derived values.
- Every control uses its Name attribute to look up the corresponding entry in BarcodeData. Only when the control's Name matches a key in the parsed barcode data will auto-fill or auto-execute behavior trigger.
- This ensures deterministic mapping between parsed barcode elements and SRC controls.
- Once a field value is consumed by a control, it is removed from BarcodeData to prevent reuse across multiple fields or screens.

API Response Structure

The MultiSegmentBarcodeApi/ParseLabel API returns a response in the following format:

```
{
  "IsMultiElementBarcode": true,
  "Data": {
    "FieldName1": {
      "Value": "<parsed value>",
      "AutoFill": true,
      "AutoExecute": false
    },
    "FieldName2": {
      "Value": "<parsed value>",
      "AutoFill": true,
      "AutoExecute": true
    }
  },
  "RawData": null
}
```

```
}
```

Auto-Fill and Auto-Execute

Each field in the parsed BarcodeData can be configured with AutoFill and AutoExecute behaviors:

- **AutoFill:** When set to true, the application automatically populates the corresponding form control with the parsed value without requiring any user interaction.
- **AutoExecute:** When set to true and AutoFill is also enabled, the application automatically triggers the Go action (form submission) after populating the field. AutoExecute will only take effect if there are no text input controls on the current page, to prevent accidental submission when manual input is still required.

BarcodeData Application in Generic View Processing

During generic-view-processing, Warehouse Mobile evaluates every input control on the screen:

- The framework inspects the Name property of each control.
- It checks if saveEntityManager.BarcodeData contains a matching key.
- If found, the value is pulled from BarcodeData
 - written into the DataModel, and
 - rendered into the corresponding UI control based on SRC configuration.

This logic is executed inside the assignBinding method of the generic-view-processor. This method is responsible for evaluating input controls, applying auto-fill rules, and determining whether auto-execute should trigger (based on control types and parsed barcode flags).

Date Format Handling (GS1 Barcodes)

GS1 barcodes encode dates in YYMMDD format (6 digits). The Warehouse Mobile application automatically detects and converts this format to the ISO date format (YYYY-MM-DD) required by the server.

For example, a GS1-encoded date of "260315" (representing March 15, 2026) will be automatically converted to "2026-03-15" before being populated in a DateTime control.

Error Handling

- If the barcode parsing API returns an error, an error toast message is displayed to the user and the scan is not processed further.
- If the parsed BarcodeData does not contain a value for the specific control where scanning occurs, the Warehouse Mobile application shows a "Scan failure" error message. This validation applies only to the control expecting input from the scan, not to every control on the page.

WAREHOUSE MOBILE GUIDE

label type of the scanned barcode does not match any configured pattern, the barcode is treated as a standard single-value scan and processed normally.

BarcodeData Lifecycle

The parsed BarcodeData is managed carefully to prevent stale data from propagating across screens or flows:

- BarcodeData is cleared when the user navigates back to the main menu.
- BarcodeData is cleared when the user presses the Cancel button on a screen.
- BarcodeData is cleared after a successful API call (when the server responds with a success message code starting with MSG_).
- Individual field values are cleared from BarcodeData as soon as they are consumed by a form control, preventing reuse.
- When multiple scans occur during the same workflow, their parsed results are appended to the existing BarcodeData rather than replacing it. This allows multi-step scanning where each scan contributes additional structured elements until the workflow or API success message clears the buffer.

Supported Control Types

The following control types support multi-segment barcode auto-population:

- Input (text input controls)
- Picker controls
- DateTime controls (with automatic GS1 date format conversion)
- Validate-Input controls

Warehouse mobile custom SRC

This feature allows SRC customizations without modifying the base SRC JSON. Clients can maintain their customizations in a separate custom format (JSON), independent of the base JSON. This ensures minimal to no impact on the client-specific customizations whenever R&D updates the base JSON file. These customizations are applied to the base SRC when the SRC is fetched from the WM UI.

The feature is controlled by the feature flag **FEATURE_36877_WM_SRC_CUSTOMIZATION**.

Configuration

The custom SRCs are configured in the same directory as the base SRC. For example, if a customer has a base SRC file in the Screen Flows folder, the client will create the corresponding custom SRC file in the same Screen Flows folder. Refer to the *Screen Flow configuration* section of this guide for details on configuring SRC files.

Example: For a custom **closeContainer.json**, when the user clicks the Close Container option, the custom screen flow is invoked. In this case if a custom JSON file configured is **CloseContainer_Custom.json**, then final JSON returned from the API will be the result of applying the custom **CloseContainer_Custom.json** on top of the base **closeContainer.json**

Identifier	Description	System Value 1	Active
Add filter value 21 RESET CLOSE			
21	custom close container	customSrcCloseContainer.json	Yes
Menu Group			
Menu Option Name Submenu Name Sequence SRC Identifier Parent Menu Resource Key Submenu Resource Key			
Menu Group: Close Container (1)			
Close Container	2	custom close container	UI_WMCLOSECONTAINER

Custom SRC format

The custom SRC supports three keywords: **ExtendSRC**, **Delete**, and **Change**.

WAREHOUSE MOBILE GUIDE

```
1 {
2   "ExtendSRC": "CloseContainer.json",
3   "Delete": [
4     {
5       "key": "$.flows[?(@.flowName == 'ContainerEntry')].Pages[?(@.Go.pageNumber == '0')].Controls[?(@.order == 1)].label"
6     }
7   ],
8   "Change": [
9     {
10      "key": "$.flows[?(@.flowName == 'ContainerEntry')].Pages[?(@.Go.pageNumber == '0')].Controls",
11      "ArrayIndex": 0,
12      "value": {
13        "label": "CONTAINERID",
14        "controltype": "label",
15        "type": "text",
16        "name": "ContainerId",
17        "ShowOnAllPage": true,
18        "visible": "true"
19      }
20    },
21    {
22      "key": "$.flows[?(@.flowName == 'ContainerEntry')].Pages[?(@.Go.pageNumber == '0')].Go.uri",
23      "value": "/outbound/scaleapi/test"
24    },
25    {
26      "key": "$.flows[?(@.flowName == 'ContainerEntry')].Title",
27      "value": "CLOSECONTAINERNew"
28    }
29  ]
30 }
31
32
33
34
```

1. ExtendSRC

- This holds the **base SRC file name** to which custom extensions to be applied.

2. Change

- This contains **key–value pairs**.
- The **key** is a **JSONPath** for the field you want to modify.

Supported actions:

a. Key–value modifications

Use this when you want to modify any value at a particular path in the base JSON.

Example – modifying a URL value

```
{
  "key": "$.flows[?(@.flowName == 'ContainerEntry')].Pages[?(@.Go.pageNumber == '0')].Go.uri",
  "value": "/outbound/scaleapi/test"
}
```

WAREHOUSE MOBILE GUIDE

Example – modifying the title

```
{
  "key": "$.flows[?(@.flowName == 'ContainerEntry')].Title",
  "value": "CLOSECONTAINERNew"
}
```

b. Adding an object to an array

Use this when you want to add an object to an array

- Example - To Add a control to an existing array of controls or add a new flow.

Provide:

- key: JSONPath of the **array**.
- value: the **object** you want to insert.
- ArrayIndex: the **position** at which to insert the object in the array.

Example – inserting a new control

```
{
  "key": "$.flows[?(@.flowName == 'ContainerEntry')].Pages[?(@.Go.pageNumber == '0')].Controls",
  "ArrayIndex": 0,
  "value": {
    "label": "CONTAINERID",
    "controltype": "label",
    "type": "text",
    "name": "ContainerId",
    "ShowOnAllPage": true,
    "visible": "true"
  }
}
```

WAREHOUSE MOBILE GUIDE

3. Delete

Use Delete to remove keys from the base JSON.

Specify a **list of keys in JSONPath format** to be deleted.

Example – deleting a label value

```
"Delete": [  
  {  
    key": "$.flows[?(@.flowName == 'ContainerEntry')].Pages[?(@.Go.pageNumber == '0')].Controls[?(@.order == 1)].label"  
  }  
],
```

Warehouse Mobile SCALE Assist

SCALE Assist is a knowledge assistant designed to provide real-time, conversational support by understanding user queries and responding with information from the product's help documentation or Customer document

In Warehouse mobile, user can access the Scale Assist feature by clicking the Scale Assist icon in the header. Since SCALE Assist is hosted as a separate application, Warehouse mobile loads it within an iframe and passes an authentication token to the Scale Assist application for user authentication.

A new component 'ChatPopoverComponent' has been created in Warehouse mobile to load the iframe. This component is displayed as a popover.

Scanning is disabled while Scale Assist is in use to prevent unintended or incorrect behaviour.

WAREHOUSE MOBILE GUIDE

Warehouse Mobile



SCALE Assist





Base Knowledge Assistant

The scaleKnowledgeBase Agent provides comprehensive information about system configuration, operational processes, and API functionalities relevant to Manhattan ACTIVE solutions. This includes details on configurable features, workflows, business rules, and integration points within the Manhattan ACTIVE suite. Users can ask questions related to setting up and customizing the system to meet specific business needs, understanding operational

Warehouse Mobile



SCALE Assist





Base Knowledge Assistant



Hello, How can I help you?
I'm currently capable of answering your questions on product functionality and configuration, my only source of training is product documentation. I'm not currently aware of context or data.

Ask a Question



Searches and their results are generated through Manhattan's AI technology. Verify results against the product documentation.